

Dataframes in Python

Technical Notes

Rodrigo Kang

Table of contents

1	Tabular Data and DataFrames	6
2	The pandas DataFrame	7
2.1	Rows as Observations and Columns as Variables	8
3	Creating DataFrames	9
3.1	From a Dictionary	9
3.2	From a List of Lists	10
3.3	From a List of Dictionaries	11
3.4	From Separate Sequences	12
3.5	Custom Row Index	13
4	Understanding DataFrame Structure	14
4.1	Shape	15
4.2	Column Names	16
4.3	Data Types	16
4.4	Information Summary	17
4.5	Inspecting the First and Last Rows	17
4.6	Descriptive Summary	18
5	Series	19
5.1	Creating a Series Directly	21
6	Selecting Columns	21
7	Selecting Rows and Columns	24
7.1	Label-Based Selection with .loc[]	24
7.2	Position-Based Selection with .iloc[]	25
7.3	Labels Are Not Positions	26
8	Slicing	28
9	Boolean Filtering	29
9.1	Filtering with .loc[]	30

9.2	Multiple Conditions	31
9.3	Membership with .isin()	32
9.4	Filtering Missing Values	33
10	Querying Versus Understanding	35
11	Creating and Transforming Columns	36
11.1	Vectorized Transformations	38
11.2	Using .assign()	39
12	Renaming Columns	40
13	Dropping Rows and Columns	41
14	Data Types and Type Conversion	42
15	Missing Data	44
15.1	Detecting Missing Values	45
15.2	Removing Missing Values	47
15.3	Filling Missing Values	48
16	Sorting Data	49
16.1	Sorting by Index	51
17	Grouping and Aggregation	51
17.1	Common Aggregations	53
17.2	Aggregating Several Variables	54
18	Multiple Aggregations	56
19	Aggregation Versus Transformation	58
20	Combining DataFrames	59
21	Concatenation	59
21.1	Column-Wise Concatenation	60
22	Merging DataFrames	61
23	Join Types	63
23.1	Inner Join	63
23.2	Left Join	64
23.3	Right Join	64
23.4	Outer Join	65
24	Keys and Cardinality	65
24.1	Validating Merge Assumptions	66
25	Row Multiplication	66

26 Indexes	68
27 Index Alignment	70
27.1 Alignment with Missing Labels	71
28 Alignment in DataFrames	72
29 Duplicates	74
30 Granularity	75
31 Data Transformation as Analytical Reasoning	76
32 Reading and Writing Tabular Data	77
32.1 Reading CSV Files	77
32.2 Controlling Import Behaviour	78
32.3 Writing CSV Files	79
33 String Data	80
33.1 Searching within Strings	82
33.2 Extracting Components	82
34 Datetime Data	83
34.1 Extracting Datetime Components	84
34.2 Time Differences	85
35 Categorical Summaries	86
36 Mapping and Replacing Values	88
37 Reshaping Tabular Data	89
37.1 Wide to Long with melt()	90
37.2 Long to Wide with pivot()	91
38 Pivot Tables	92
39 Relational Data	93
40 Primary and Foreign Keys	94
41 SQL and pandas	94
42 SQLite	95
43 Reading SQL Queries into DataFrames	95
44 Filtering in SQL	96
45 Aggregation in SQL	97

46 SQL Joins	98
47 Joining Order Details and Products	99
48 Combining Several Relational Tables	100
49 SQL or pandas?	101
50 DataFrames and Machine Learning	102
51 Categorical Features	103
52 From DataFrame to NumPy	104
53 Avoiding Target Leakage	105
54 Train/Test Contamination	106
55 Common DataFrame Problems	106
55.1 Unintended Row Multiplication	107
55.2 Unexpected Missing Values after Alignment	107
55.3 Incorrect Data Types	107
55.4 Silent Changes in Grain	107
55.5 Chained Assignment	108
55.6 Row-Wise Iteration	108
56 A Systematic DataFrame Workflow	109
57 Key Takeaways	109
58 Problems	111
58.1 Problem 1 — Creating a DataFrame	111
58.2 Problem 2 — Series Versus DataFrame	112
58.3 Problem 3 — Structural Inspection	112
58.4 Problem 4 — Label-Based and Position-Based Selection	113
58.5 Problem 5 — Slicing with .loc[] and .iloc[]	114
58.6 Problem 6 — Boolean Filtering	114
58.7 Problem 7 — Combining Boolean Conditions	115
58.8 Problem 8 — Membership Filtering	116
58.9 Problem 9 — Creating a Derived Numerical Column	116
58.10 Problem 10 — Conditional Columns	117
58.11 Problem 11 — .assign() and Method Chaining	118
58.12 Problem 12 — Renaming and Dropping Columns	118
58.13 Problem 13 — Type Conversion	119
58.14 Problem 14 — Detecting Missing Values	119
58.15 Problem 15 — Dropping Missing Values	120
58.16 Problem 16 — Simple Imputation	121
58.17 Problem 17 — Sorting	121
58.18 Problem 18 — Frequency Tables	121

58.19	Problem 19 — String Cleaning	122
58.20	Problem 20 — String Filtering	122
58.21	Problem 21 — Splitting Structured Text	123
58.22	Problem 22 — Datetime Components	123
58.23	Problem 23 — Time Differences	124
58.24	Problem 24 — GroupBy and Mean	124
58.25	Problem 25 — Multiple Group Aggregations	125
58.26	Problem 26 — Aggregation Changes Grain	126
58.27	Problem 27 — Group-Level Transformation	126
58.28	Problem 28 — Concatenating Observations	127
58.29	Problem 29 — Horizontal Concatenation and Index Alignment	127
58.30	Problem 30 — Basic Merge	128
58.31	Problem 31 — Join Types	129
58.32	Problem 32 — Cardinality	129
58.33	Problem 33 — Unexpected Row Multiplication	130
58.34	Problem 34 — Index Alignment in Series	130
58.35	Problem 35 — Missing Labels During Alignment	131
58.36	Problem 36 — Duplicate Rows Versus Repeated Keys	132
58.37	Problem 37 — Wide to Long Format	132
58.38	Problem 38 — Long to Wide Format	133
58.39	Problem 39 — Pivot Table with Aggregation	133
58.40	Problem 40 — Reading a CSV	134
58.41	Problem 41 — Writing a DataFrame	135
58.42	Problem 42 — Northwind Schema Reasoning	135
58.43	Problem 43 — Northwind Customers Query	135
58.44	Problem 44 — Filtering in SQL Versus pandas	136
58.45	Problem 45 — Northwind Orders and Customers	136
58.46	Problem 46 — Northwind Order Lines	137
58.47	Problem 47 — Northwind Multi-Table Join	137
58.48	Problem 48 — SQL Aggregation	138
58.49	Problem 49 — SQL or pandas?	138
58.50	Problem 50 — Preparing Predictors and Target	139
58.51	Problem 51 — One-Hot Encoding	140
58.52	Problem 52 — DataFrame to NumPy	140
58.53	Problem 53 — Chained Assignment	141
58.54	Problem 54 — Vectorization Versus Row Iteration	141
58.55	Problem 55 — Diagnose a Merge Bug	141
58.56	Problem 56 — Diagnose an Alignment Bug	142
58.57	Problem 57 — Data Leakage Through Imputation	143
58.58	Problem 58 — Leakage Through Aggregation	143
58.59	Problem 59 — Grain Reasoning	144
58.60	Problem 60 — Integrated DataFrame Reasoning	144

1 Tabular Data and DataFrames

A large proportion of applied data analysis begins with **tabular data**.

In tabular form, data are organized into rows and columns:

- each **row** usually represents an observation;
- each **column** usually represents a variable;
- each cell contains the value of a variable for a particular observation.

For example, consider a small dataset containing information about four observations:

id	age	income	group
1	25	42000	A
2	31	51000	B
3	28	47000	A
4	40	68000	B

The rows correspond to individual observations, while the columns describe their characteristics.

If there are n observations and p variables, the data can be viewed conceptually as an $n \times p$ rectangular structure.

However, real tabular datasets differ from ordinary numerical matrices in an important way: different columns may represent different types of information.

For example:

- age may contain integers;
- income may contain floating-point values;
- group may contain strings;
- a date column may contain datetime values;
- another column may contain Boolean values.

This heterogeneous structure is one reason why NumPy arrays alone are not always the most convenient representation for tabular data.

In Python, the pandas library provides the **DataFrame** abstraction for working with this type of data.

The conventional import is:

```

1 import pandas as pd
2 import numpy as np

```

2 The pandas DataFrame

A DataFrame is a mutable two-dimensional labelled data structure.

Its two axes are:

- the **row index**;
- the **column index**.

A simple DataFrame can be created from a dictionary:

```

1 data = {
2     "x": [1, 2, 3, 4, 5],
3     "y": [2, 4, 6, 8, 10]
4 }
5
6 df = pd.DataFrame(data)
7
8 df

```

	x	y
0	1	2
1	2	4
2	3	6
3	4	8
4	5	10

The dictionary keys become column labels:

```

1 df.columns

```

```
Index(['x', 'y'], dtype='object')
```

while pandas automatically creates a row index:

```
1 df.index
```

```
RangeIndex(start=0, stop=5, step=1)
```

The DataFrame therefore contains both the underlying values and labels describing their position.

This distinction is fundamental.

A DataFrame is not simply a two-dimensional NumPy array with prettier output. Its labelled axes influence:

- selection;
- alignment;
- aggregation;
- joining;
- transformation.

These behaviours become particularly important when working with real datasets.

2.1 Rows as Observations and Columns as Variables

In most analytical datasets, the conventional interpretation is:

row $\longrightarrow$ observation,

and:

column $\longrightarrow$ variable.

For example:

```
1 people = pd.DataFrame({
2     "age": [25, 31, 28, 40],
3     "income": [42000, 51000, 47000, 68000],
4     "group": ["A", "B", "A", "B"]
5 })
6
7 people
```

	age	income	group
0	25	42000	A
1	31	51000	B
2	28	47000	A
3	40	68000	B

Each row represents one person and each column describes one characteristic.

This structure appears throughout Data Science and Machine Learning.

A supervised learning dataset is often conceptually separated into:

X = predictor variables

and:

y = target variable.

Both commonly originate from columns of a DataFrame before being passed to a modelling library.

3 Creating DataFrames

DataFrames can be created from several Python data structures.

The most useful representations depend on how the original data are organized.

3.1 From a Dictionary

A dictionary of lists is a natural representation when each dictionary entry corresponds to a variable:

```

1 data = {
2     "x": [1, 2, 3, 4, 5],
3     "y": [2, 4, 6, 8, 10]
4 }
5
6 df = pd.DataFrame(data)
```

```
7
8 df
```

	x	y
0	1	2
1	2	4
2	3	6
3	4	8
4	5	10

The keys:

```
1 "x"
2 "y"
```

become column names.

The corresponding lists provide the column values.

All columns must be compatible with the number of rows being constructed.

3.2 From a List of Lists

A DataFrame can also be constructed from records represented as lists:

```
1 data = [
2     [1, 2],
3     [2, 4],
4     [3, 6],
5     [4, 8],
6     [5, 10]
7 ]
8
9 df = pd.DataFrame(
10     data,
11     columns=["x", "y"]
12 )
13
14 df
```

	x	y
0	1	2
1	2	4
2	3	6
3	4	8
4	5	10

Here, each inner list represents a row.

Because the data themselves do not contain variable names, the column labels are supplied separately.

3.3 From a List of Dictionaries

Another common representation is a list of records:

```

1 data = [
2     {"x": 1, "y": 2},
3     {"x": 2, "y": 4},
4     {"x": 3, "y": 6},
5     {"x": 4, "y": 8},
6     {"x": 5, "y": 10}
7 ]
8
9 df = pd.DataFrame(data)
10
11 df

```

	x	y
0	1	2
1	2	4
2	3	6
3	4	8
4	5	10

Each dictionary represents an observation, while the dictionary keys identify the variables.

This record-oriented structure is conceptually close to formats such as JSON.

3.4 From Separate Sequences

Suppose the variables already exist as separate sequences:

```
1 x = [1, 2, 3, 4, 5]
2 y = [2, 4, 6, 8, 10]
```

They can be combined directly:

```
1 df = pd.DataFrame({
2     "x": x,
3     "y": y
4 })
5
6 df
```

	x	y
0	1	2
1	2	4
2	3	6
3	4	8
4	5	10

The built-in `zip()` function can also combine the sequences into row-oriented records:

```
1 data = list(
2     zip(x, y)
3 )
4
5 data
```

```
[(1, 2), (2, 4), (3, 6), (4, 8), (5, 10)]
```

and then:

```
1 df = pd.DataFrame(
2     data,
3     columns=["x", "y"]
4 )
5
6 df
```

	x	y
0	1	2
1	2	4
2	3	6
3	4	8
4	5	10

Both approaches produce the same tabular structure, but the dictionary representation often communicates the column semantics more directly.

3.5 Custom Row Index

A custom row index can be supplied during construction:

```

1 df = pd.DataFrame(
2     {
3         "x": [1, 2, 3, 4, 5],
4         "y": [2, 4, 6, 8, 10]
5     },
6     index=[
7         "obs1",
8         "obs2",
9         "obs3",
10        "obs4",
11        "obs5"
12    ]
13 )
14
15 df

```

	x	y
obs1	1	2
obs2	2	4
obs3	3	6
obs4	4	8
obs5	5	10

The labels:

```
1 obs1
2 obs2
3 obs3
4 obs4
5 obs5
```

now identify the rows.

An index is not necessarily an ordinary data variable. It is part of the DataFrame's labelling structure.

Whether a particular identifier should remain a regular column or become an index depends on how the data will be used.

4 Understanding DataFrame Structure

Before analysing an unfamiliar dataset, it is useful to inspect its basic structure.

Consider:

```
1 df = pd.DataFrame({
2     "age": [25, 31, 28, 40, 36],
3     "income": [
4         42000.0,
5         51000.0,
6         47000.0,
7         68000.0,
8         59000.0
9     ],
10    "group": [
11        "A",
12        "B",
13        "A",
14        "B",
15        "A"
16    ],
17    "active": [
18        True,
19        True,
20        False,
21        True,
```

```

22     False
23 ]
24 })
25
26 df

```

	age	income	group	active
0	25	42000.0	A	True
1	31	51000.0	B	True
2	28	47000.0	A	False
3	40	68000.0	B	True
4	36	59000.0	A	False

4.1 Shape

The `.shape` attribute returns:

```
1 df.shape
```

(5, 4)

The result has the form:

```
1 (number of rows, number of columns)
```

If:

```
1 df.shape == (5, 4)
```

then the DataFrame contains five observations and four variables.

The number of dimensions can also be inspected using:

```
1 df.ndim
```

2

A DataFrame is two-dimensional.

4.2 Column Names

The column labels are available through:

```
1 df.columns
```

```
Index(['age', 'income', 'group', 'active'], dtype='object')
```

The row labels are available through:

```
1 df.index
```

```
RangeIndex(start=0, stop=5, step=1)
```

These labels are part of the DataFrame structure and are used extensively in pandas operations.

4.3 Data Types

Column data types can be inspected using:

```
1 df.dtypes
```

```
age      int64
income   float64
group     object
active    bool
dtype: object
```

Different columns may have different data types.

For example:

- age may be an integer;
- income may be floating point;
- group may contain strings;
- active may be Boolean.

This differs from an ordinary NumPy array, where all elements generally share a common dtype.

4.4 Information Summary

The `.info()` method provides a compact structural summary:

```
1 df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 4 columns):
#   Column  Non-Null Count  Dtype
---  -
0  age      5 non-null      int64
1  income   5 non-null      float64
2  group    5 non-null      object
3  active   5 non-null      bool
dtypes: bool(1), float64(1), int64(1), object(1)
memory usage: 257.0+ bytes
```

It reports information such as:

- number of rows;
- column names;
- number of non-null values;
- data types;
- approximate memory usage.

This is often one of the first commands used when examining an unfamiliar DataFrame.

4.5 Inspecting the First and Last Rows

The first rows can be displayed using:

```
1 df.head()
```

	age	income	group	active
0	25	42000.0	A	True
1	31	51000.0	B	True
2	28	47000.0	A	False
3	40	68000.0	B	True
4	36	59000.0	A	False

or, for a specific number of rows:

```
1 df.head(3)
```

	age	income	group	active
0	25	42000.0	A	True
1	31	51000.0	B	True
2	28	47000.0	A	False

Similarly:

```
1 df.tail()
```

	age	income	group	active
0	25	42000.0	A	True
1	31	51000.0	B	True
2	28	47000.0	A	False
3	40	68000.0	B	True
4	36	59000.0	A	False

displays the final rows.

These operations are useful for checking:

- variable names;
- approximate data structure;
- unexpected values;
- whether an import operation behaved as expected.

They do not replace a systematic analysis of the data, but they provide a quick structural inspection.

4.6 Descriptive Summary

For numerical columns:

```
1 df.describe()
```

	age	income
count	5.000000	5.000000
mean	32.000000	53400.000000
std	6.041523	10261.578826
min	25.000000	42000.000000
25%	28.000000	47000.000000
50%	31.000000	51000.000000
75%	36.000000	59000.000000
max	40.000000	68000.000000

returns common descriptive statistics such as:

- count;
- mean;
- standard deviation;
- minimum;
- quartiles;
- maximum.

These summaries are useful, but they compress the original data.

For example, two variables can have similar means and standard deviations while having substantially different distributions.

Descriptive statistics should therefore often be combined with the graphical analysis considered previously.

5 Series

A pandas **Series** is a one-dimensional labelled data structure.

A DataFrame column is usually represented as a Series.

For example:

```
1 age = df["age"]
2
3 age
```

```
0  25
1  31
2  28
3  40
4  36
Name: age, dtype: int64
```

Its type is:

```
1 type(age)
```

```
pandas.core.series.Series
```

The Series contains:

- values;
- an index;
- optionally, a name.

These can be inspected using:

```
1 age.index
```

```
RangeIndex(start=0, stop=5, step=1)
```

```
1 age.name
```

```
'age'
```

and:

```
1 age.dtype
```

```
dtype('int64')
```

A useful conceptual relationship is:

DataFrame $\approx$ collection of aligned Series.

The word **aligned** is important because pandas uses index labels when combining many objects.

This behaviour will be considered in more detail later.

5.1 Creating a Series Directly

A Series can also be created independently:

```
1 scores = pd.Series(  
2     [72, 85, 91, 68],  
3     index=[  
4         "A",  
5         "B",  
6         "C",  
7         "D"  
8     ],  
9     name="score"  
10 )  
11  
12 scores
```

```
A    72  
B    85  
C    91  
D    68  
Name: score, dtype: int64
```

The index labels identify the observations.

A Series therefore differs conceptually from an ordinary one-dimensional NumPy array because positions are accompanied by labels.

6 Selecting Columns

Selecting data is one of the most common DataFrame operations.

Consider:

```
1 df = pd.DataFrame({  
2     "name": [  
3         "Alice",  
4         "Bob",  
5         "Carol",  
6         "David",  
7         "Eva"
```

```

8 ],
9 "age": [
10     25,
11     31,
12     28,
13     40,
14     36
15 ],
16 "income": [
17     42000,
18     51000,
19     47000,
20     68000,
21     59000
22 ],
23 "group": [
24     "A",
25     "B",
26     "A",
27     "B",
28     "A"
29 ]
30 })
31
32 df

```

	name	age	income	group
0	Alice	25	42000	A
1	Bob	31	51000	B
2	Carol	28	47000	A
3	David	40	68000	B
4	Eva	36	59000	A

A single column can be selected using:

```
1 df["age"]
```

```

0  25
1  31

```

```
2 28
3 40
4 36
Name: age, dtype: int64
```

The result is a Series.

This can be verified using:

```
1 type(df["age"])
```

```
pandas.core.series.Series
```

Several columns can be selected by providing a list of column names:

```
1 df[
2   ["age", "income"]
3 ]
```

	age	income
0	25	42000
1	31	51000
2	28	47000
3	40	68000
4	36	59000

The result is a DataFrame:

```
1 type(
2   df[["age", "income"]]
3 )
```

```
pandas.core.frame.DataFrame
```

This distinction is useful:

```
1 df["age"]
```

returns a one-dimensional Series, whereas:

```
1 df[["age"]]
```

returns a two-dimensional DataFrame containing one column.

7 Selecting Rows and Columns

Pandas provides two central indexers for explicit selection:

- `.loc[]`;
- `.iloc[]`.

The distinction between them is fundamental.

7.1 Label-Based Selection with `.loc[]`

`.loc[]` selects data using **labels**.

For example:

```
1 df.loc[0]
```

```
name    Alice
age      25
income  42000
group    A
Name: 0, dtype: object
```

returns the row whose index label is 0.

Rows and columns can be selected simultaneously:

```
1 df.loc[
2     0,
3     "income"
4 ]
```

```
np.int64(42000)
```

This selects the value at row label 0 and column "income".

Several rows and columns can also be selected:

```
1 df.loc[
2     [0, 2, 4],
3     ["name", "income"]
4 ]
```

	name	income
0	Alice	42000
2	Carol	47000
4	Eva	59000

The general structure is:

```
1 df.loc[
2     row_labels,
3     column_labels
4 ]
```

7.2 Position-Based Selection with .iloc[]

.iloc[] selects data according to **integer position**.

For example:

```
1 df.iloc[0]
```

```
name    Alice
age      25
income  42000
group    A
Name: 0, dtype: object
```

returns the first row, regardless of its label.

A single value can be selected using:

```
1 df.iloc[
2     0,
3     2
4 ]
```

`np.int64(42000)`

This means:

- first row;
- third column.

Several positions can be selected:

```
1 df.iloc[
2     [0, 2, 4],
3     [0, 2]
4 ]
```

	name	income
0	Alice	42000
2	Carol	47000
4	Eva	59000

The general structure is:

```
1 df.iloc[
2     row_positions,
3     column_positions
4 ]
```

7.3 Labels Are Not Positions

The distinction becomes clearer when the DataFrame has a custom index.

```

1 indexed_df = df.copy()
2
3 indexed_df.index = [
4     10,
5     20,
6     30,
7     40,
8     50
9 ]
10
11 indexed_df

```

	name	age	income	group
10	Alice	25	42000	A
20	Bob	31	51000	B
30	Carol	28	47000	A
40	David	40	68000	B
50	Eva	36	59000	A

Now:

```
1 indexed_df.loc[20]
```

```

name    Bob
age      31
income  51000
group    B
Name: 20, dtype: object

```

selects the row whose **label** is 20.

By contrast:

```
1 indexed_df.iloc[1]
```

```

name    Bob
age      31
income  51000
group    B
Name: 20, dtype: object

```

selects the **second row**.

Both expressions return the same row in this example, but they do so for fundamentally different reasons.

Confusing labels and positions is a common source of errors when working with pandas.

8 Slicing

Both `.loc[]` and `.iloc[]` support slicing, but their endpoint behaviour differs.

With `.iloc[]`, ordinary Python positional slicing rules apply:

```
1 df.iloc[
2     1:4
3 ]
```

	name	age	income	group
1	Bob	31	51000	B
2	Carol	28	47000	A
3	David	40	68000	B

The row at position 4 is excluded.

Thus the slice contains positions:

```
1 1, 2, 3
```

With `.loc[]`, label slices generally include the final label:

```
1 df.loc[
2     1:3
3 ]
```

	name	age	income	group
1	Bob	31	51000	B
2	Carol	28	47000	A
3	David	40	68000	B

This includes labels:

```
1 1, 2, 3
```

This difference reflects the distinction between positional slicing and label-based slicing.

9 Boolean Filtering

DataFrames are frequently filtered according to logical conditions.

Suppose we want observations whose income exceeds 50,000.

The comparison:

```
1 df["income"] > 50000
```

```
0  False
1   True
2  False
3   True
4   True
Name: income, dtype: bool
```

returns a Boolean Series.

Each row receives either:

```
1 True
```

or:

```
1 False
```

depending on whether the condition is satisfied.

This Boolean Series can be used as a mask:

```
1 df[
2     df["income"] > 50000
3 ]
```

	name	age	income	group
1	Bob	31	51000	B
3	David	40	68000	B
4	Eva	36	59000	A

Only rows for which the condition evaluates to True are retained.

This operation can be understood conceptually as:

DataFrame + Boolean condition $\longrightarrow$ subset of observations.

9.1 Filtering with .loc[]

The same filtering operation can be written explicitly using .loc[]:

```
1 df.loc[
2     df["income"] > 50000
3 ]
```

	name	age	income	group
1	Bob	31	51000	B
3	David	40	68000	B
4	Eva	36	59000	A

A useful advantage is that row filtering and column selection can be combined:

```
1 df.loc[
2     df["income"] > 50000,
3     ["name", "income"]
4 ]
```

	name	income
1	Bob	51000
3	David	68000
4	Eva	59000

This expression means:

1. retain rows where income exceeds 50,000;
2. return only the name and income columns.

9.2 Multiple Conditions

Conditions can be combined using:

- & for element-wise AND;
- | for element-wise OR;
- ~ for element-wise NOT.

Suppose we want observations where:

age > 30

and:

income > 50000.

We can write:

```
1 df.loc[
2     (df["age"] > 30)
3     & (df["income"] > 50000)
4 ]
```

	name	age	income	group
1	Bob	31	51000	B
3	David	40	68000	B
4	Eva	36	59000	A

Parentheses around each comparison are important because of Python operator precedence.

For an OR condition:

```
1 df.loc[
2     (df["group"] == "A")
3     | (df["income"] > 60000)
4 ]
```

	name	age	income	group
0	Alice	25	42000	A
2	Carol	28	47000	A
3	David	40	68000	B
4	Eva	36	59000	A

Negation can be written using:

```
1 df.loc[
2     ~(df["group"] == "A")
3 ]
```

	name	age	income	group
1	Bob	31	51000	B
3	David	40	68000	B

although an explicit inequality may be clearer in simple cases:

```
1 df.loc[
2     df["group"] != "A"
3 ]
```

	name	age	income	group
1	Bob	31	51000	B
3	David	40	68000	B

9.3 Membership with .isin()

Suppose observations should be retained when a column belongs to a collection of accepted values.

For example:

```
1 df.loc[
2     df["name"].isin(
3         ["Alice", "Carol", "Eva"]
4     )
5 ]
```

	name	age	income	group
0	Alice	25	42000	A
2	Carol	28	47000	A
4	Eva	36	59000	A

This is usually clearer than combining several equality comparisons with `|`.

The corresponding negation is:

```
1 df.loc[
2     ~df["name"].isin(
3         ["Alice", "Carol", "Eva"]
4     )
5 ]
```

	name	age	income	group
1	Bob	31	51000	B
3	David	40	68000	B

9.4 Filtering Missing Values

Missing values require special treatment because comparisons involving missing data do not behave like ordinary equality comparisons.

Pandas provides:

```
1 .isna()
```

and:

```
1 .notna()
```

for identifying missing and non-missing observations.

For example:

```

1 df_missing = pd.DataFrame({
2     "name": [
3         "Alice",
4         "Bob",
5         "Carol",
6         "David"
7     ],
8     "score": [
9         82.0,
10        np.nan,
11        91.0,
12        np.nan
13    ]
14 })
15
16 df_missing

```

	name	score
0	Alice	82.0
1	Bob	NaN
2	Carol	91.0
3	David	NaN

Rows with missing scores can be selected using:

```

1 df_missing.loc[
2     df_missing["score"].isna()
3 ]

```

	name	score
1	Bob	NaN
3	David	NaN

Rows with observed scores can be selected using:

```

1 df_missing.loc[
2     df_missing["score"].notna()
3 ]

```

	name	score
0	Alice	82.0
2	Carol	91.0

Missing data will be considered in greater detail later because deciding how to handle them is an analytical question, not merely a syntactic operation.

10 Querying Versus Understanding

Pandas makes complex filtering expressions compact.

For example:

```

1 df.loc[
2     (df["age"] >= 30)
3     & (df["income"] < 65000)
4     & df["group"].isin(["A", "B"]),
5     ["name", "age", "income"]
6 ]

```

	name	age	income
1	Bob	31	51000
4	Eva	36	59000

However, compact syntax should not obscure the underlying logic.

A useful way to reason about a filtering operation is:

1. **What population does each row represent?**
2. **What condition determines whether an observation is retained?**
3. **Which variables are required in the resulting dataset?**
4. **How should missing values affect the condition?**

The objective is not simply to produce a syntactically valid expression. It is to ensure that the resulting subset corresponds to the intended analytical population.

This distinction becomes particularly important in Data Science and Machine Learning, where an incorrect filter can silently change:

- the training population;
- class balance;
- temporal coverage;
- group representation;
- estimated model performance.

DataFrame manipulation is therefore part of the analytical reasoning process, not merely data-processing syntax.

11 Creating and Transforming Columns

Data analysis frequently requires constructing new variables from existing ones.

Consider:

```
1 df = pd.DataFrame({
2     "product": [
3         "A",
4         "B",
5         "C",
6         "D"
7     ],
8     "price": [
9         10.0,
10        15.0,
11        8.0,
12        20.0
13    ],
14    "quantity": [
15        3,
16        2,
17        5,
18        4
19    ]
20 })
```

21

22

df

	product	price	quantity
0	A	10.0	3
1	B	15.0	2
2	C	8.0	5
3	D	20.0	4

A new column can be created by assignment:

```

1 df["revenue"] = (
2     df["price"]
3     * df["quantity"]
4 )
5
6 df

```

	product	price	quantity	revenue
0	A	10.0	3	30.0
1	B	15.0	2	30.0
2	C	8.0	5	40.0
3	D	20.0	4	80.0

The operation is applied element by element:

$$r_i = p_i q_i,$$

where:

- p_i is the price of observation i ;
- q_i is its quantity;
- r_i is the resulting revenue.

This illustrates an important property of pandas: operations between compatible Series are generally **vectorized**.

Instead of explicitly iterating through the rows, the transformation is expressed at the level of entire columns.

11.1 Vectorized Transformations

Suppose prices must be increased by 10%.

We can write:

```
1 df["adjusted_price"] = (  
2     df["price"] * 1.10  
3 )  
4  
5 df
```

	product	price	quantity	revenue	adjusted_price
0	A	10.0	3	30.0	11.0
1	B	15.0	2	30.0	16.5
2	C	8.0	5	40.0	8.8
3	D	20.0	4	80.0	22.0

Similarly, a Boolean variable can be constructed from a logical condition:

```
1 df["high_value"] = (  
2     df["revenue"] >= 50  
3 )  
4  
5 df
```

	product	price	quantity	revenue	adjusted_price	high_value
0	A	10.0	3	30.0	11.0	False
1	B	15.0	2	30.0	16.5	False
2	C	8.0	5	40.0	8.8	False
3	D	20.0	4	80.0	22.0	True

The resulting column contains Boolean values.

A categorical variable can also be constructed conditionally:

```

1 df["revenue_category"] = np.where(
2     df["revenue"] >= 50,
3     "High",
4     "Standard"
5 )
6
7 df

```

	product	price	quantity	revenue	adjusted_price	high_value	revenue_category
0	A	10.0	3	30.0	11.0	False	Standard
1	B	15.0	2	30.0	16.5	False	Standard
2	C	8.0	5	40.0	8.8	False	Standard
3	D	20.0	4	80.0	22.0	True	High

The general principle is:

When a transformation can be expressed naturally as an operation on complete columns, a vectorized expression is usually clearer than explicit row-by-row iteration.

11.2 Using .assign()

New columns can also be created with .assign():

```

1 df = df.assign(
2     unit_tax=df["price"] * 0.15
3 )
4
5 df

```

	product	price	quantity	revenue	adjusted_price	high_value	revenue_category	unit_tax
0	A	10.0	3	30.0	11.0	False	Standard	1.50
1	B	15.0	2	30.0	16.5	False	Standard	2.25
2	C	8.0	5	40.0	8.8	False	Standard	1.20
3	D	20.0	4	80.0	22.0	True	High	3.00

One advantage of .assign() is that it fits naturally into method chains.

For example:

```

1 result = (
2     df
3     .assign(
4         total_cost=lambda x:
5             x["price"] * x["quantity"]
6     )
7     .sort_values(
8         "total_cost",
9         ascending=False
10    )
11 )
12
13 result

```

	product	price	quantity	revenue	adjusted_price	high_value	revenue_category	unit_tax	total_
3	D	20.0	4	80.0	22.0	True	High	3.00	80.0
2	C	8.0	5	40.0	8.8	False	Standard	1.20	40.0
1	B	15.0	2	30.0	16.5	False	Standard	2.25	30.0
0	A	10.0	3	30.0	11.0	False	Standard	1.50	30.0

Method chaining can make a sequence of transformations easier to read when each step represents a clear analytical operation.

However, long chains can also become difficult to inspect or debug.

The objective is not to maximize compactness, but to express the transformation clearly.

12 Renaming Columns

Column names often need to be changed because:

- imported names are unclear;
- naming conventions are inconsistent;
- a variable has changed meaning after transformation;
- names contain spaces or punctuation that make later code awkward.

Columns can be renamed using:

```

1 df = df.rename(
2     columns={
3         "price": "unit_price",
4         "quantity": "units"
5     }
6 )
7
8 df

```

	product	unit_price	units	revenue	adjusted_price	high_value	revenue_category	unit_tax
0	A	10.0	3	30.0	11.0	False	Standard	1.50
1	B	15.0	2	30.0	16.5	False	Standard	2.25
2	C	8.0	5	40.0	8.8	False	Standard	1.20
3	D	20.0	4	80.0	22.0	True	High	3.00

A useful naming convention should make variables understandable without requiring unnecessary interpretation.

For analytical code, names such as:

```

1 unit_price
2 order_date
3 customer_id
4 total_revenue

```

are generally more informative than abbreviated names whose meaning is not obvious.

13 Dropping Rows and Columns

Columns can be removed using `.drop()`:

```

1 df_reduced = df.drop(
2     columns=[
3         "revenue_category"
4     ]
5 )
6
7 df_reduced

```

	product	unit_price	units	revenue	adjusted_price	high_value	unit_tax
0	A	10.0	3	30.0	11.0	False	1.50
1	B	15.0	2	30.0	16.5	False	2.25
2	C	8.0	5	40.0	8.8	False	1.20
3	D	20.0	4	80.0	22.0	True	3.00

Rows can also be removed by index label:

```
1 df.drop(
2   index=[0]
3 )
```

	product	unit_price	units	revenue	adjusted_price	high_value	revenue_category	unit_tax
1	B	15.0	2	30.0	16.5	False	Standard	2.25
2	C	8.0	5	40.0	8.8	False	Standard	1.20
3	D	20.0	4	80.0	22.0	True	High	3.00

Dropping data should be treated as an analytical decision.

Removing a column because it is genuinely irrelevant is different from removing it because its meaning is not yet understood.

Similarly, removing observations can alter:

- sample size;
- class balance;
- group representation;
- estimated distributions;
- downstream model performance.

The operation is syntactically simple, but its analytical consequences may not be.

14 Data Types and Type Conversion

The meaning of a column and its pandas data type should be compatible.

Consider:

```

1 data = pd.DataFrame({
2     "id": [
3         "1",
4         "2",
5         "3",
6         "4"
7     ],
8     "value": [
9         "10.5",
10        "12.7",
11        "9.8",
12        "15.1"
13    ]
14 })
15
16 data.dtypes

```

```

id    object
value object
dtype: object

```

Both columns may initially be represented as strings.

The numerical column can be converted using:

```

1 data["value"] = (
2     data["value"]
3     .astype(float)
4 )
5
6 data.dtypes

```

```

id    object
value float64
dtype: object

```

Pandas also provides conversion functions such as:

```

1 pd.to_numeric()
2 pd.to_datetime()

```

For example:

```
1 dates = pd.DataFrame({
2     "date": [
3         "2026-01-01",
4         "2026-01-15",
5         "2026-02-01"
6     ]
7 })
8
9 dates["date"] = pd.to_datetime(
10     dates["date"]
11 )
12
13 dates.dtypes
```

```
date    datetime64[ns]
dtype: object
```

Correct data types matter because they determine which operations are meaningful.

A date stored as an ordinary string cannot be manipulated as naturally as a datetime variable.

Likewise, a numerical value stored as text can interfere with:

- arithmetic;
- sorting;
- aggregation;
- statistical modelling.

15 Missing Data

Real datasets frequently contain missing values.

In pandas, missing numerical values are commonly represented using NaN, although modern pandas supports several missing-value representations depending on the data type.

Consider:

```

1 df_missing = pd.DataFrame({
2     "age": [
3         25,
4         31,
5         np.nan,
6         40,
7         36
8     ],
9     "income": [
10        42000,
11        np.nan,
12        47000,
13        68000,
14        59000
15    ],
16    "group": [
17        "A",
18        "B",
19        "A",
20        None,
21        "A"
22    ]
23 })
24
25 df_missing

```

	age	income	group
0	25.0	42000.0	A
1	31.0	NaN	B
2	NaN	47000.0	A
3	40.0	68000.0	None
4	36.0	59000.0	A

15.1 Detecting Missing Values

Missing values can be identified using:

```
1 df_missing.isna()
```

	age	income	group
0	False	False	False
1	False	True	False
2	True	False	False
3	False	False	True
4	False	False	False

The result is a Boolean DataFrame.

The number of missing values in each column can be calculated using:

```
1 df_missing.isna().sum()
```

```
age      1
income   1
group    1
dtype: int64
```

This works because Boolean values can be aggregated numerically:

True $\rightarrow$ 1, False $\rightarrow$ 0.

The complementary operation is:

```
1 df_missing.notna()
```

	age	income	group
0	True	True	True
1	True	False	True
2	False	True	True
3	True	True	False
4	True	True	True

15.2 Removing Missing Values

Rows containing missing values can be removed using:

```
1 df_missing.dropna()
```

	age	income	group
0	25.0	42000.0	A
4	36.0	59000.0	A

A subset of columns can determine whether a row is removed:

```
1 df_missing.dropna(  
2     subset=["income"]  
3 )
```

	age	income	group
0	25.0	42000.0	A
2	NaN	47000.0	A
3	40.0	68000.0	None
4	36.0	59000.0	A

This retains observations for which income is available, regardless of missing values elsewhere.

However:

Missing observations should not be removed automatically simply because `dropna()` makes the operation easy.

The consequences depend on **why the values are missing** and how the resulting dataset will be used.

Dropping incomplete observations can:

- reduce sample size;
- alter distributions;
- disproportionately remove particular groups;
- introduce bias.

15.3 Filling Missing Values

Missing values can be replaced using `.fillna()`.

For example:

```
1 df_filled = df_missing.copy()
2
3 df_filled["income"] = (
4     df_filled["income"]
5     .fillna(
6         df_filled["income"].median()
7     )
8 )
9
10 df_filled
```

	age	income	group
0	25.0	42000.0	A
1	31.0	53000.0	B
2	NaN	47000.0	A
3	40.0	68000.0	None
4	36.0	59000.0	A

This replaces missing income values with the observed median.

Although syntactically straightforward, this is an **imputation decision**.

Median imputation changes the data distribution and does not preserve the uncertainty associated with the missing values.

The appropriate treatment of missing data depends on:

- the mechanism producing missingness;
- the variable involved;
- the analytical objective;
- the downstream model.

Pandas provides the operations, but it does not determine the correct statistical treatment.

16 Sorting Data

Rows can be ordered according to one or more variables using `.sort_values()`.

Consider:

```
1 df = pd.DataFrame({
2     "name": [
3         "Alice",
4         "Bob",
5         "Carol",
6         "David",
7         "Eva"
8     ],
9     "age": [
10        25,
11        31,
12        28,
13        40,
14        36
15    ],
16    "income": [
17        42000,
18        51000,
19        47000,
20        68000,
21        59000
22    ],
23    "group": [
24        "A",
25        "B",
26        "A",
27        "B",
28        "A"
29    ]
30 })
```

Sorting by income:

```

1 df.sort_values(
2     "income"
3 )

```

	name	age	income	group
0	Alice	25	42000	A
2	Carol	28	47000	A
1	Bob	31	51000	B
4	Eva	36	59000	A
3	David	40	68000	B

Descending order can be requested using:

```

1 df.sort_values(
2     "income",
3     ascending=False
4 )

```

	name	age	income	group
3	David	40	68000	B
4	Eva	36	59000	A
1	Bob	31	51000	B
2	Carol	28	47000	A
0	Alice	25	42000	A

Several variables can determine the ordering:

```

1 df.sort_values(
2     by=[
3         "group",
4         "income"
5     ],
6     ascending=[
7         True,
8         False
9     ]
10 )

```

	name	age	income	group
4	Eva	36	59000	A
2	Carol	28	47000	A
0	Alice	25	42000	A
3	David	40	68000	B
1	Bob	31	51000	B

This first orders observations by group and then orders observations within each group by decreasing income.

16.1 Sorting by Index

Rows can also be sorted according to their index:

```
1 df.sort_index()
```

	name	age	income	group
0	Alice	25	42000	A
1	Bob	31	51000	B
2	Carol	28	47000	A
3	David	40	68000	B
4	Eva	36	59000	A

Sorting does not generally change the underlying observations. It changes their order of presentation.

Nevertheless, ordering can matter when:

- inspecting results;
- constructing reports;
- working with temporal observations;
- calculating operations that depend on sequence.

17 Grouping and Aggregation

Many analytical questions concern **groups of observations** rather than individual rows.

Examples include:

- average income by region;
- total sales by product;
- number of customers by country;
- mean model error by class;
- monthly revenue;
- number of transactions by customer.

Pandas provides the `groupby()` mechanism for this purpose.

Consider:

```

1 sales = pd.DataFrame({
2     "region": [
3         "North",
4         "North",
5         "South",
6         "South",
7         "South",
8         "West"
9     ],
10    "sales": [
11        100,
12        150,
13        120,
14        180,
15        200,
16        140
17    ]
18 })
19
20 sales

```

	region	sales
0	North	100
1	North	150
2	South	120
3	South	180
4	South	200
5	West	140

The mean sales by region can be calculated using:

```

1 sales.groupby(
2     "region"
3 )["sales"].mean()

```

```

region
North    125.000000
South    166.666667
West     140.000000
Name: sales, dtype: float64

```

Conceptually, this operation performs three steps:

1. **split** observations into groups;
2. **apply** an operation within each group;
3. **combine** the results.

This is often described as the:

split–apply–combine strategy.

If the observations are partitioned into groups:

$$G_1, G_2, \dots, G_k,$$

then an aggregation applies a function f independently:

$$G_j \longrightarrow f(G_j).$$

For a mean:

$$\bar{x}_j = \frac{1}{n_j} \sum_{i \in G_j} x_i.$$

17.1 Common Aggregations

Common aggregation methods include:

```
1 .mean()
2 .sum()
3 .count()
4 .size()
5 .min()
6 .max()
7 .median()
8 .std()
```

For example:

```
1 sales.groupby(
2     "region"
3 )["sales"].sum()
```

```
region
North    250
South    500
West     140
Name: sales, dtype: int64
```

returns total sales by region.

The number of rows in each group can be obtained with:

```
1 sales.groupby(
2     "region"
3 ).size()
```

```
region
North    2
South    3
West     1
dtype: int64
```

17.2 Aggregating Several Variables

Consider:

```

1 employees = pd.DataFrame({
2     "department": [
3         "A",
4         "A",
5         "B",
6         "B",
7         "B",
8         "C"
9     ],
10    "salary": [
11        60000,
12        65000,
13        70000,
14        72000,
15        68000,
16        75000
17    ],
18    "experience": [
19        2,
20        4,
21        5,
22        6,
23        3,
24        8
25    ]
26 })

```

Several numerical variables can be aggregated simultaneously:

```

1 employees.groupby(
2     "department"
3 )[
4     ["salary", "experience"]
5 ].mean()

```

	salary	experience
department		
A	62500.0	3.000000
B	70000.0	4.666667

	salary	experience
department		
C	75000.0	8.000000

The result contains one row per department.

This illustrates an important change in **granularity**.

Before aggregation:

one row $\rightarrow$ one employee.

After aggregation:

one row $\rightarrow$ one department.

Understanding the granularity represented by each row is essential when manipulating tabular data.

18 Multiple Aggregations

The `.agg()` method allows several statistics to be calculated together:

```

1 employees.groupby(
2     "department"
3 )["salary"].agg([
4     "count",
5     "mean",
6     "median",
7     "min",
8     "max"
9 ])

```

	count	mean	median	min	max
department					
A	2	62500.0	62500.0	60000	65000
B	3	70000.0	70000.0	68000	72000

	count	mean	median	min	max
department					
C	1	75000.0	75000.0	75000	75000

Named aggregations can produce more explicit output:

```

1 summary = (
2     employees
3     .groupby("department")
4     .agg(
5         employee_count=(
6             "salary",
7             "size"
8         ),
9         mean_salary=(
10            "salary",
11            "mean"
12        ),
13        median_salary=(
14            "salary",
15            "median"
16        ),
17        mean_experience=(
18            "experience",
19            "mean"
20        )
21    )
22 )
23
24 summary

```

	employee_count	mean_salary	median_salary	mean_experience
department				
A	2	62500.0	62500.0	3.000000
B	3	70000.0	70000.0	4.666667
C	1	75000.0	75000.0	8.000000

This form is particularly useful when the resulting table will be used later because the output variable names communicate their meaning directly.

19 Aggregation Versus Transformation

Aggregation reduces several observations to a smaller number of group-level results.

Sometimes, however, we need a group statistic while preserving the original number of rows.

Consider:

```
1 employees["department_mean_salary"] = (  
2     employees  
3     .groupby("department")["salary"]  
4     .transform("mean")  
5 )  
6  
7 employees
```

	department	salary	experience	department_mean_salary
0	A	60000	2	62500.0
1	A	65000	4	62500.0
2	B	70000	5	70000.0
3	B	72000	6	70000.0
4	B	68000	3	70000.0
5	C	75000	8	75000.0

Each employee now receives the mean salary of their department.

The number of rows has not changed.

This distinction is fundamental:

```
1 aggregation  
2   many rows → fewer rows  
3  
4 transformation  
5   many rows → same number of rows
```

For example, salary relative to the department mean can be calculated as:

```

1 employees["salary_difference"] = (
2     employees["salary"]
3     - employees["department_mean_salary"]
4 )
5
6 employees

```

	department	salary	experience	department_mean_salary	salary_difference
0	A	60000	2	62500.0	-2500.0
1	A	65000	4	62500.0	2500.0
2	B	70000	5	70000.0	0.0
3	B	72000	6	70000.0	2000.0
4	B	68000	3	70000.0	-2000.0
5	C	75000	8	75000.0	0.0

This type of group-relative feature construction is common in feature engineering.

20 Combining DataFrames

Analytical information is frequently distributed across multiple tables.

Pandas provides several mechanisms for combining tabular data.

Two important operations are:

- **concatenation;**
- **merging.**

They solve different problems.

21 Concatenation

`pd.concat()` combines DataFrames along an axis.

Consider two batches of observations:

```

1 df_a = pd.DataFrame({
2     "id": [1, 2],
3     "value": [10, 20]
4 })
5
6 df_b = pd.DataFrame({
7     "id": [3, 4],
8     "value": [30, 40]
9 })

```

They can be stacked vertically:

```

1 combined = pd.concat(
2     [df_a, df_b],
3     ignore_index=True
4 )
5
6 combined

```

	id	value
0	1	10
1	2	20
2	3	30
3	4	40

The operation can be understood as:

$$[A] + [B] \longrightarrow \begin{bmatrix} A \\ B \end{bmatrix}.$$

This is appropriate when both DataFrames represent the same variables for different observations.

21.1 Column-Wise Concatenation

DataFrames can also be concatenated horizontally:

```

1 left = pd.DataFrame({
2     "x": [1, 2, 3]
3 })
4
5 right = pd.DataFrame({
6     "y": [10, 20, 30]
7 })
8
9 pd.concat(
10     [left, right],
11     axis=1
12 )

```

	x	y
0	1	10
1	2	20
2	3	30

However, horizontal concatenation uses the index to align rows.

This means that it should not be interpreted as simply placing columns next to each other without considering labels.

22 Merging DataFrames

`merge()` combines DataFrames according to one or more **keys**.

Consider:

```

1 customers = pd.DataFrame({
2     "customer_id": [
3         1,
4         2,
5         3
6     ],
7     "customer_name": [
8         "Alpha",
9         "Beta",
10        "Gamma"

```

```

11     ]
12 })
13
14 orders = pd.DataFrame({
15     "order_id": [
16         101,
17         102,
18         103,
19         104
20     ],
21     "customer_id": [
22         1,
23         2,
24         1,
25         3
26     ],
27     "amount": [
28         50,
29         80,
30         30,
31         120
32     ]
33 })

```

The tables contain different information but share:

```
1 customer_id
```

They can be merged using:

```

1 customer_orders = orders.merge(
2     customers,
3     on="customer_id"
4 )
5
6 customer_orders

```

	order_id	customer_id	amount	customer_name
0	101	1	50	Alpha
1	102	2	80	Beta
2	103	1	30	Alpha
3	104	3	120	Gamma

The resulting table combines information from both DataFrames according to matching key values.

This operation is conceptually similar to a relational database join.

23 Join Types

The how parameter controls which keys are retained.

23.1 Inner Join

```

1 orders.merge(
2     customers,
3     on="customer_id",
4     how="inner"
5 )

```

	order_id	customer_id	amount	customer_name
0	101	1	50	Alpha
1	102	2	80	Beta
2	103	1	30	Alpha
3	104	3	120	Gamma

An inner join retains only keys that match in both tables.

If:

$$K_L$$

is the set of keys in the left table and:

$$K_R$$

the set in the right table, then an inner join retains:

$$K_L \cap K_R.$$

23.2 Left Join

```
1 orders.merge(  
2     customers,  
3     on="customer_id",  
4     how="left"  
5 )
```

	order_id	customer_id	amount	customer_name
0	101	1	50	Alpha
1	102	2	80	Beta
2	103	1	30	Alpha
3	104	3	120	Gamma

A left join preserves all rows from the left table.

When no corresponding key exists in the right table, columns contributed by the right table contain missing values.

23.3 Right Join

A right join preserves all rows from the right table:

```
1 left.merge(  
2     right,  
3     how="right"  
4 )
```

23.4 Outer Join

An outer join retains keys appearing in either table:

$$K_L \cup K_R.$$

In pandas:

```
1 left.merge(  
2     right,  
3     how="outer"  
4 )
```

Unmatched information is represented using missing values.

24 Keys and Cardinality

A merge is not defined only by the column names used as keys.

The **cardinality** of the relationship matters.

Common relationships include:

- one-to-one;
- one-to-many;
- many-to-one;
- many-to-many.

Consider the customer and order example.

Each customer appears once in customers, but a customer may appear several times in orders.

Therefore:

Customers $\rightarrow$ Orders

is a one-to-many relationship.

When the tables are merged, customer information is repeated for each matching order.

This repetition is not necessarily duplication in the sense of a data-quality error. It follows from the granularity of the resulting table:

one row $\rightarrow$ one order.

24.1 Validating Merge Assumptions

Pandas can validate expected merge cardinality.

For example:

```
1 customer_orders = orders.merge(  
2     customers,  
3     on="customer_id",  
4     how="left",  
5     validate="many_to_one"  
6 )  
7  
8 customer_orders
```

	order_id	customer_id	amount	customer_name
0	101	1	50	Alpha
1	102	2	80	Beta
2	103	1	30	Alpha
3	104	3	120	Gamma

This states that:

- many rows may contain the same `customer_id` in orders;
- each `customer_id` should occur at most once in customers.

If the assumption is violated, pandas raises an error.

This can be extremely useful because unintended duplicate keys can silently multiply rows after a merge.

25 Row Multiplication

Suppose both tables contain repeated values for the merge key.

Then a many-to-many merge can create several combinations for the same key.

For example:

```

1 left = pd.DataFrame({
2     "key": ["A", "A"],
3     "x": [1, 2]
4 })
5
6 right = pd.DataFrame({
7     "key": ["A", "A", "A"],
8     "y": [10, 20, 30]
9 })
10
11 left.merge(
12     right,
13     on="key"
14 )

```

	key	x	y
0	A	1	10
1	A	1	20
2	A	1	30
3	A	2	10
4	A	2	20
5	A	2	30

There are:

2

matching rows on the left and:

3

on the right.

The result therefore contains:

$$2 \times 3 = 6$$

rows for key A.

This behaviour is correct according to relational join semantics.

The analytical question is whether such a relationship was intended.

For this reason, after important merges it is useful to inspect:

- row counts;
- key uniqueness;
- unmatched keys;
- expected cardinality.

26 Indexes

Every DataFrame has an index.

Consider:

```
1 df = pd.DataFrame({
2     "customer_id": [
3         101,
4         102,
5         103
6     ],
7     "name": [
8         "Alpha",
9         "Beta",
10        "Gamma"
11    ],
12    "value": [
13        10,
14        20,
15        30
16    ]
17 })
18
19 df
```

	customer_id	name	value
0	101	Alpha	10
1	102	Beta	20
2	103	Gamma	30

customer_id	name	value
-------------	------	-------

The default index is:

```
1 df.index
```

RangeIndex(start=0, stop=3, step=1)

A column can be moved into the index using:

```
1 indexed = df.set_index(
2     "customer_id"
3 )
4
5 indexed
```

	name	value
customer_id		
101	Alpha	10
102	Beta	20
103	Gamma	30

Now:

```
1 indexed.loc[102]
```

```
name    Beta
value    20
Name: 102, dtype: object
```

selects the observation whose customer identifier is 102.

The index can be converted back into a regular column:

```
1 indexed.reset_index()
```

	customer_id	name	value
0	101	Alpha	10
1	102	Beta	20
2	103	Gamma	30

Indexes can be useful for:

- labelled selection;
- alignment;
- time-series operations;
- identifying observations.

However, not every identifier should automatically become an index.

Keeping identifiers as ordinary columns is often clearer when they are:

- merge keys;
- model features;
- exported to external systems;
- manipulated explicitly.

27 Index Alignment

One of the most important pandas behaviours is **automatic alignment by index label**.

Consider:

```

1 a = pd.Series(
2     [10, 20, 30],
3     index=[
4         "A",
5         "B",
6         "C"
7     ]
8 )
9
10 b = pd.Series(
11     [1, 2, 3],
12     index=[
13         "C",

```

```

14     "B",
15     "A"
16 ]
17 )

```

Now calculate:

```

1 a + b

```

```

A    13
B    22
C    31
dtype: int64

```

Pandas does not simply add values according to physical position.

Instead, it aligns the Series by their labels.

The operation is therefore:

$$a_A + b_A,$$

$$a_B + b_B,$$

$$a_C + b_C.$$

This is fundamentally different from ordinary positional NumPy arithmetic.

27.1 Alignment with Missing Labels

Consider:

```

1 a = pd.Series(
2     [10, 20, 30],
3     index=[
4         "A",
5         "B",
6         "C"
7     ]

```

```

8 )
9
10 b = pd.Series(
11     [1, 2, 3],
12     index=[
13         "B",
14         "C",
15         "D"
16     ]
17 )

```

Then:

```

1 a + b

```

```

A   NaN
B   21.0
C   32.0
D   NaN
dtype: float64

```

produces missing values where one of the labels is absent.

Only labels present in both Series can be added directly.

This behaviour is powerful because it reduces the risk of combining values belonging to different observations merely because they occupy the same physical position.

However, it can also produce unexpected missing values if index alignment is not understood.

28 Alignment in DataFrames

The same principle applies when DataFrames are combined or assigned.

Consider:

```

1 df = pd.DataFrame(
2     {
3         "value": [
4             100,
5             200,

```

```

6         300
7     ]
8 },
9     index=[
10         "A",
11         "B",
12         "C"
13     ]
14 )
15
16 adjustment = pd.Series(
17     [10, 20, 30],
18     index=[
19         "C",
20         "A",
21         "B"
22     ]
23 )
24
25 df["adjustment"] = adjustment
26
27 df

```

	value	adjustment
A	100	20
B	200	30
C	300	10

The values are assigned according to matching index labels rather than according to their original order in the Series.

This is one of the defining features of pandas.

A useful mental model is:

NumPy primarily reasons about array positions and shapes; pandas additionally reasons about labels.

Both abstractions are useful, but they solve different problems.

29 Duplicates

Duplicate rows can be detected using:

```
1 duplicate_data = pd.DataFrame({
2     "id": [
3         1,
4         2,
5         2,
6         3
7     ],
8     "value": [
9         10,
10        20,
11        20,
12        30
13    ]
14 })
15
16 duplicate_data.duplicated()
```

```
0  False
1  False
2   True
3  False
dtype: bool
```

They can be removed using:

```
1 duplicate_data.drop_duplicates()
```

	id	value
0	1	10
1	2	20
3	3	30

Duplicates can also be defined relative to selected columns:

```
1 duplicate_data.duplicated(  
2     subset=["id"]  
3 )
```

```
0 False  
1 False  
2 True  
3 False  
dtype: bool
```

However, repeated values are not automatically duplicate observations.

For example, in an order table a customer identifier may legitimately appear many times because one customer can place many orders.

Whether repeated values indicate duplication depends on the expected **granularity** and **key structure** of the table.

This connects directly with merge cardinality.

30 Granularity

One of the most useful questions when working with DataFrames is:

What does one row represent?

Possible answers include:

- one customer;
- one transaction;
- one product;
- one customer per month;
- one measurement at one time point;
- one model prediction;
- one order line.

This property is often called the **granularity** or **grain** of the dataset.

Many DataFrame errors can be understood as failures to keep track of grain.

For example:

- `groupby()` changes grain;
- aggregation changes grain;

- a one-to-many merge may preserve the grain of the many-side table;
- a many-to-many merge may create a new and unexpected grain;
- `drop_duplicates()` can destroy legitimate observations if repeated keys are expected.

Before and after an important transformation, it is useful to ask:

1. What does one row represent?
2. Which columns uniquely identify a row?
3. Has the operation changed that interpretation?
4. Is the resulting number of rows plausible?

These questions are often more important than remembering the exact pandas method used to perform the operation.

31 Data Transformation as Analytical Reasoning

Pandas makes transformations concise, but a valid operation is not necessarily a valid analytical operation.

For example:

```
1 df.groupby(...)
2 df.merge(...)
3 df.dropna(...)
4 df.drop_duplicates(...)
```

can all execute successfully while producing a dataset that does not represent the intended population.

A robust DataFrame workflow therefore combines syntax with explicit reasoning about:

- data types;
- missingness;
- keys;
- cardinality;
- granularity;
- aggregation;
- alignment.

These concepts become particularly important when DataFrames are constructed from relational databases, where the structure of the source tables explicitly encodes relationships between entities.

The next step is therefore to connect pandas with **external tabular data and relational databases**.

32 Reading and Writing Tabular Data

In practice, DataFrames are usually created from external data rather than manually constructed inside Python.

Common sources include:

- CSV files;
- spreadsheets;
- relational databases;
- columnar storage formats;
- application APIs;
- other analytical systems.

Pandas provides functions that convert many of these external representations into DataFrames.

The general pattern is:

external data source $\longrightarrow$ DataFrame $\longrightarrow$ analysis or transformation.

The DataFrame acts as an in-memory tabular representation on which pandas operations can be performed.

32.1 Reading CSV Files

A CSV file can be loaded using:

```
1 df = pd.read_csv(  
2     "data.csv"  
3 )
```

The resulting object is a DataFrame:

```
1 type(df)
```

After loading an unfamiliar file, useful first checks include:

```
1 df.head()
```

```
1 df.shape
```

```
1 df.info()
```

```
1 df.isna().sum()
```

These operations help answer basic questions:

- Were the columns interpreted correctly?
- How many observations were loaded?
- Which data types were inferred?
- Are missing values present?
- Does the imported structure match expectations?

Reading a file successfully does not guarantee that it has been interpreted correctly.

32.2 Controlling Import Behaviour

Real files may require additional information during import.

For example:

```
1 df = pd.read_csv(  
2     "data.csv",  
3     usecols=[  
4         "customer_id",  
5         "order_date",  
6         "amount"  
7     ]  
8 )
```

selects only the required columns.

Missing-value conventions can also be specified:

```

1 df = pd.read_csv(
2     "data.csv",
3     na_values=[
4         "",
5         "NA",
6         "missing"
7     ]
8 )

```

Date parsing can be performed after loading:

```

1 df["order_date"] = pd.to_datetime(
2     df["order_date"]
3 )

```

The important principle is that **data import is part of data interpretation**.

Choices made during import determine:

- data types;
- missing-value representations;
- columns available downstream;
- sometimes the apparent values themselves.

32.3 Writing CSV Files

A DataFrame can be exported using:

```

1 df.to_csv(
2     "output.csv",
3     index=False
4 )

```

Using:

```

1 index=False

```

prevents the DataFrame index from being written as an additional column when that index is not part of the intended dataset.

Before exporting, it is useful to ask whether:

- the index represents meaningful data;
- column names are appropriate;
- data types will survive the format;
- important metadata may be lost.

CSV is widely compatible but stores relatively little information about types and metadata.

33 String Data

Text columns frequently require transformations before they can be analysed or used as model inputs.

Pandas provides vectorized string operations through the `.str` accessor.

Consider:

```

1 customers = pd.DataFrame({
2     "name": [
3         " Alice Smith ",
4         "BOB JONES",
5         "Carol Brown"
6     ],
7     "city": [
8         "Auckland",
9         "WELLINGTON",
10        "christchurch"
11    ]
12 })
13
14 customers
```

	name	city
0	Alice Smith	Auckland
1	BOB JONES	WELLINGTON
2	Carol Brown	christchurch

Whitespace can be removed using:

```

1 customers["name"] = (
2     customers["name"]
3     .str.strip()
4 )
5
6 customers

```

	name	city
0	Alice Smith	Auckland
1	BOB JONES	WELLINGTON
2	Carol Brown	christchurch

Text can be converted to lower case:

```

1 customers["city_lower"] = (
2     customers["city"]
3     .str.lower()
4 )
5
6 customers

```

	name	city	city_lower
0	Alice Smith	Auckland	auckland
1	BOB JONES	WELLINGTON	wellington
2	Carol Brown	christchurch	christchurch

or title case:

```

1 customers["city_clean"] = (
2     customers["city"]
3     .str.title()
4 )
5
6 customers

```

	name	city	city_lower	city_clean
0	Alice Smith	Auckland	auckland	Auckland
1	BOB JONES	WELLINGTON	wellington	Wellington
2	Carol Brown	christchurch	christchurch	Christchurch

These operations are vectorized over the Series.

33.1 Searching within Strings

String values can be tested using operations such as:

```
1 .str.contains()
2 .str.startswith()
3 .str.endswith()
```

For example:

```
1 customers.loc[
2     customers["name"].str.contains(
3         "Alice",
4         case=False,
5         na=False
6     )
7 ]
```

	name	city	city_lower	city_clean
0	Alice Smith	Auckland	auckland	Auckland

The result of `.str.contains()` is a Boolean Series that can be used as a filter.

33.2 Extracting Components

Suppose identifiers follow a structured pattern:

```

1 data = pd.DataFrame({
2     "code": [
3         "NZ-AKL-001",
4         "NZ-WLG-002",
5         "NZ-CHC-003"
6     ]
7 })

```

The code can be split:

```

1 data[
2     ["country", "city", "id"]
3 ] = (
4     data["code"]
5     .str.split(
6         "-",
7         expand=True
8     )
9 )
10
11 data

```

	code	country	city	id
0	NZ-AKL-001	NZ	AKL	001
1	NZ-WLG-002	NZ	WLG	002
2	NZ-CHC-003	NZ	CHC	003

String manipulation therefore allows information encoded inside text fields to be converted into explicit variables.

However, transformations should reflect actual semantics rather than merely convenient patterns.

34 Datetime Data

Dates and times frequently appear in transactional, operational, and machine learning datasets.

A string such as:

```
1 2026-08-18
```

may visually resemble a date while still being stored as text.

Datetime conversion can be performed using:

```
1 events = pd.DataFrame({
2     "event_date": [
3         "2026-01-15",
4         "2026-02-03",
5         "2026-03-21"
6     ]
7 })
8
9 events["event_date"] = pd.to_datetime(
10     events["event_date"]
11 )
12
13 events.dtypes
```

```
event_date    datetime64[ns]
dtype: object
```

Once a Series has a datetime dtype, pandas provides the `.dt` accessor.

34.1 Extracting Datetime Components

For example:

```
1 events["year"] = (
2     events["event_date"].dt.year
3 )
4
5 events["month"] = (
6     events["event_date"].dt.month
7 )
8
9 events["day"] = (
10     events["event_date"].dt.day
11 )
```

12

13 events

	event_date	year	month	day
0	2026-01-15	2026	1	15
1	2026-02-03	2026	2	3
2	2026-03-21	2026	3	21

Other useful components include:

```

1 .dt.dayofweek
2 .dt.day_name()
3 .dt.month_name()
4 .dt.quarter

```

These can support exploratory analysis and feature construction.

34.2 Time Differences

Datetime values can be subtracted:

```

1 dates = pd.DataFrame({
2     "start": pd.to_datetime([
3         "2026-01-01",
4         "2026-02-01"
5     ]),
6     "end": pd.to_datetime([
7         "2026-01-10",
8         "2026-02-20"
9     ])
10 })
11
12 dates["duration"] = (
13     dates["end"]
14     - dates["start"]
15 )
16
17 dates

```

	start	end	duration
0	2026-01-01	2026-01-10	9 days
1	2026-02-01	2026-02-20	19 days

The result is a timedelta.

The duration in days can be obtained with:

```

1 dates["duration_days"] = (
2     dates["duration"].dt.days
3 )
4
5 dates
```

	start	end	duration	duration_days
0	2026-01-01	2026-01-10	9 days	9
1	2026-02-01	2026-02-20	19 days	19

Datetime operations are particularly important when observations have a temporal structure.

In machine learning, temporal information also requires care because random train/test splitting may be inappropriate when future observations must be predicted from past data.

35 Categorical Summaries

For categorical variables, frequency-based methods provide useful summaries.

Consider:

```

1 customers = pd.DataFrame({
2     "country": [
3         "NZ",
4         "AU",
5         "NZ",
6         "US",
7         "NZ",
8         "AU"
9     ]
10 })
```

Frequencies can be calculated using:

```
1 customers["country"].value_counts()
```

```
country
NZ    3
AU    2
US    1
Name: count, dtype: int64
```

Relative frequencies can be obtained using:

```
1 customers[
2     "country"
3 ].value_counts(
4     normalize=True
5 )
```

```
country
NZ    0.500000
AU    0.333333
US    0.166667
Name: proportion, dtype: float64
```

The number of unique observed values is:

```
1 customers["country"].nunique()
```

```
3
```

and the unique values themselves can be inspected using:

```
1 customers["country"].unique()
```

```
array(['NZ', 'AU', 'US'], dtype=object)
```

These operations are useful when checking:

- categorical distributions;
- unexpected levels;
- class imbalance;
- high-cardinality variables.

36 Mapping and Replacing Values

Known values can be mapped to alternative representations.

For example:

```
1 status = pd.DataFrame({
2     "code": [
3         0,
4         1,
5         1,
6         0
7     ]
8 })
```

A mapping can be defined:

```
1 status["label"] = (
2     status["code"]
3     .map({
4         0: "inactive",
5         1: "active"
6     })
7 )
8
9 status
```

	code	label
0	0	inactive
1	1	active
2	1	active
3	0	inactive

`.map()` is useful when a Series value has a direct mapping to another value.

Values can also be replaced using:

```
1 status["code"].replace({
2     0: False,
3     1: True
4 })
```

```
C:\Users\Rodrigo\AppData\Local\Temp\ipykernel_4224\1683683675.py:1: FutureWarning: Downcasting behavior in `replace` is depr
status["code"].replace({
```

```
0 False
1 True
2 True
3 False
Name: code, dtype: bool
```

The appropriate operation depends on whether the task represents:

- recoding;
- lookup;
- transformation;
- cleaning.

37 Reshaping Tabular Data

The same information can often be represented using different table layouts.

Two common structures are **wide** and **long** formats.

Consider:

```
1 wide = pd.DataFrame({
2     "customer": [
3         "A",
4         "B",
5         "C"
6     ],
7     "sales_2024": [
8         100,
9         150,
10        120
11    ],
12    "sales_2025": [
13        130,
14        160,
15        140
16    ]
17 })
```

wide

	customer	sales_2024	sales_2025
0	A	100	130
1	B	150	160
2	C	120	140

Each year appears as a separate column.

This is a wide representation.

A long representation might instead contain:

customer | year | sales

with multiple rows per customer.

37.1 Wide to Long with melt()

Pandas can reshape the table using:

```
long = wide.melt(  
    id_vars="customer",  
    value_vars=[  
        "sales_2024",  
        "sales_2025"  
    ],  
    var_name="year",  
    value_name="sales"  
)  
long
```

	customer	year	sales
0	A	sales_2024	100
1	B	sales_2024	150
2	C	sales_2024	120

	customer	year	sales
3	A	sales_2025	130
4	B	sales_2025	160
5	C	sales_2025	140

The grain changes from:

one row per customer

to:

one row per customer-year.

This illustrates why reshaping should be understood in terms of data meaning, not only table appearance.

37.2 Long to Wide with pivot()

The long representation can be reshaped:

```

1 long.pivot(
2     index="customer",
3     columns="year",
4     values="sales"
5 )

```

	year	sales_2024	sales_2025
customer			
A		100	130
B		150	160
C		120	140

`pivot()` reorganizes existing values without aggregation.

If multiple rows exist for the same combination of index and column keys, a simple pivot is ambiguous.

38 Pivot Tables

When several observations correspond to the same combination of groups, aggregation may be required.

Consider:

```
1 sales = pd.DataFrame({
2     "region": [
3         "North",
4         "North",
5         "North",
6         "South",
7         "South",
8         "South"
9     ],
10    "product": [
11        "A",
12        "A",
13        "B",
14        "A",
15        "B",
16        "B"
17    ],
18    "amount": [
19        100,
20        120,
21        80,
22        140,
23        110,
24        130
25    ]
26 })
```

A pivot table can calculate mean sales for each region-product combination:

```
1 sales.pivot_table(
2     index="region",
3     columns="product",
4     values="amount",
```

```

5 aggfunc="mean"
6 )

```

product	A	B
region		
North	110.0	80.0
South	140.0	120.0

The operation combines:

- grouping;
- aggregation;
- reshaping.

A pivot table therefore changes both the numerical summary and the layout of the data.

39 Relational Data

Tabular data are frequently stored in relational databases rather than in a single flat table.

A relational database separates information into tables representing different entities and relationships.

The Northwind schema provides a useful example.

Its Customers table stores customer information, while Orders contains a CustomerID foreign key linking each order to a customer. Orders also links to Employees and Shippers. OrderDetails links individual order lines to both Orders and Products.

This structure separates different grains:

```

1 Customers
2   one row → one customer
3
4 Orders
5   one row → one order
6
7 OrderDetails
8   one row → one order line

```

Products
one row → one product

This is precisely why keys and cardinality matter.

40 Primary and Foreign Keys

A **primary key** uniquely identifies a row within a table.

In Northwind:

CustomerID

is the primary key of Customers, and:

OrderID

is the primary key of Orders. :contentReferenceoacite:2 :contentReferenceoacite:3

A **foreign key** references a key in another table.

For example, Orders.CustomerID references Customers.CustomerID. Similarly, OrderDetails.OrderID references Orders.OrderID, while OrderDetails.ProductID references Products.ProductID. :contentReferenceoacite:4

These relationships encode how separate tables can be combined.

41 SQL and pandas

SQL and pandas provide overlapping but different mechanisms for working with tabular data.

SQL operates naturally where the data are stored in a relational database.

Pandas operates naturally once data have been loaded into Python memory.

A common workflow is:

database → SQL query → DataFrame → analysis.

The boundary between SQL and pandas should be chosen deliberately.

Filtering and aggregation that can be performed efficiently inside a database may be preferable to loading an unnecessarily large raw table into memory.

42 SQLite

Python includes support for SQLite through the standard-library `sqlite3` module.

A connection to a Northwind SQLite database can be created using:

```
1 import sqlite3
2
3 connection = sqlite3.connect(
4     "Northwind.db"
5 )
```

The SQL script supplied for these notes creates the Northwind tables in SQLite and can also be executed directly using the SQLite command-line interface. [:contentReferenceoacite:5](#)

Once a connection exists, pandas can execute SQL queries and load the result directly into a DataFrame.

43 Reading SQL Queries into DataFrames

For example:

```
1 query = """
2 SELECT
3     CustomerID,
4     CustomerName,
5     Country
6 FROM Customers
7 """
8
9 customers = pd.read_sql_query(
10     query,
11     connection
12 )
```

The resulting customers object is an ordinary DataFrame.

Pandas operations can then be applied:

```
1 customers.head()
```

```
1 customers[
2     "Country"
3 ].value_counts()
```

SQL determines which data are returned from the database; pandas determines how the resulting in-memory table is manipulated afterward.

44 Filtering in SQL

Suppose only customers from a particular country are required.

The filtering can happen inside the SQL query:

```
1 query = """
2 SELECT
3     CustomerID,
4     CustomerName,
5     Country
6 FROM Customers
7 WHERE Country = 'Germany'
8 """
```

rather than loading the full customer table and then applying:

```
1 customers.loc[
2     customers["Country"] == "Germany"
3 ]
```

Both approaches can produce the same analytical subset.

The difference is **where the operation occurs**.

For small datasets, the distinction may have little practical effect.

For large databases, filtering before transferring the data can substantially reduce:

- memory consumption;
- network transfer;
- processing inside Python.

45 Aggregation in SQL

SQL can also perform group-level aggregation.

For example:

```
1 query = """
2 SELECT
3     Country,
4     COUNT(*) AS customer_count
5 FROM Customers
6 GROUP BY Country
7 ORDER BY customer_count DESC
8 """
```

The result can be loaded directly:

```
1 customer_counts = pd.read_sql_query(
2     query,
3     connection
4 )
```

The equivalent conceptual pandas workflow would be based on:

```
1 customers.groupby(
2     "Country"
3 ).size()
```

The syntax differs, but both operations express the same analytical idea:

group observations → aggregate within groups.

46 SQL Joins

The relationships defined in Northwind make joins particularly useful.

For example, each order contains a CustomerID, while customer names and countries are stored separately in Customers. :contentReferenceoaicite:6 :contentReferenceoaicite:7

A SQL join can combine them:

```
1 query = ""
2 SELECT
3     o.OrderID,
4     o.OrderDate,
5     c.CustomerName,
6     c.Country
7 FROM Orders AS o
8 JOIN Customers AS c
9     ON o.CustomerID = c.CustomerID
10 ""
```

Loading the query:

```
1 orders = pd.read_sql_query(
2     query,
3     connection
4 )
```

produces a DataFrame whose grain remains:

one row per order.

Customer information is repeated across the orders belonging to the same customer because the relationship is:

one customer → many orders.

47 Joining Order Details and Products

Northwind also provides a more granular relationship.

OrderDetails contains OrderID, ProductID, and Quantity, while Products contains the corresponding product information and price. :contentReferenceoaicite:8

A query can construct order-line information:

```
1 query = """
2 SELECT
3     od.OrderID,
4     od.ProductID,
5     p.ProductName,
6     od.Quantity,
7     p.Price
8 FROM OrderDetails AS od
9 JOIN Products AS p
10    ON od.ProductID = p.ProductID
11 """
```

The resulting DataFrame has grain:

one row per order line.

A derived value can then be constructed in pandas:

```
1 order_lines["line_value"] = (
2     order_lines["Quantity"]
3     * order_lines["Price"]
4 )
```

This example demonstrates a realistic division of responsibilities:

- SQL resolves relational structure;
- pandas performs subsequent analytical transformations.

48 Combining Several Relational Tables

A richer query can combine:

- orders;
- customers;
- order details;
- products.

Conceptually:

```
1 Customers
2   ↓ CustomerID
3 Orders
4   ↓ OrderID
5 OrderDetails
6   ↓ ProductID
7 Products
```

For example:

```
1 query = ""
2 SELECT
3     o.OrderID,
4     o.OrderDate,
5     c.CustomerName,
6     c.Country,
7     p.ProductName,
8     od.Quantity,
9     p.Price
10 FROM Orders AS o
11 JOIN Customers AS c
12     ON o.CustomerID = c.CustomerID
13 JOIN OrderDetails AS od
14     ON o.OrderID = od.OrderID
15 JOIN Products AS p
16     ON od.ProductID = p.ProductID
17 ""
```

The resulting grain is no longer one row per order.

Because one order may contain several order details, the result represents:

one row per order line.

This illustrates a critical principle:

A join can change the number of rows and the meaning of each row even when every join condition is technically correct.

49 SQL or pandas?

There is no universal rule stating that transformation should happen exclusively in SQL or exclusively in pandas.

SQL is often preferable for:

- selecting required rows;
- selecting required columns;
- joining large relational tables;
- aggregating data before transfer;
- leveraging database query engines.

Pandas is often convenient for:

- exploratory transformations;
- feature engineering;
- statistical analysis;
- integration with Python libraries;
- model preparation;
- complex in-memory workflows.

A common practical strategy is:

Use SQL to retrieve an appropriately shaped analytical dataset, then use pandas for analysis and modelling transformations.

The appropriate boundary depends on:

- data volume;
- database capabilities;
- workflow architecture;
- maintainability;
- reproducibility.

50 DataFrames and Machine Learning

Pandas DataFrames are commonly used during the stages immediately preceding model fitting.

Consider:

```
1 df = pd.DataFrame({
2     "age": [
3         25,
4         31,
5         28,
6         40,
7         36
8     ],
9     "income": [
10        42000,
11        51000,
12        47000,
13        68000,
14        59000
15    ],
16    "segment": [
17        "A",
18        "B",
19        "A",
20        "B",
21        "A"
22    ],
23    "target": [
24        0,
25        1,
26        0,
27        1,
28        1
29    ]
30 })
```

The predictors and target can be separated:

```

1 X = df[
2     [
3         "age",
4         "income",
5         "segment"
6     ]
7 ]
8
9 y = df["target"]

```

Here:

```

1 X

```

is a DataFrame, while:

```

1 y

```

is a Series.

The conceptual structure is:

$X \in$ tabular feature space,

and:

$y \in$ target space.

51 Categorical Features

Many machine learning algorithms ultimately require numerical input.

A categorical column such as:

```

1 segment

```

therefore cannot always be passed directly to an estimator.

Pandas can produce indicator variables using:

```

1 X_encoded = pd.get_dummies(
2     X,
3     columns=[
4         "segment"
5     ],
6     dtype=int
7 )
8
9 X_encoded

```

	age	income	segment_A	segment_B
0	25	42000	1	0
1	31	51000	0	1
2	28	47000	1	0
3	40	68000	0	1
4	36	59000	1	0

The categorical variable is replaced by binary indicator columns.

This is commonly called **one-hot encoding**.

However, encoding is not simply a formatting step.

Its appropriateness depends on:

- the meaning of the category;
- the model being used;
- category cardinality;
- how unseen categories will be handled later.

In production modelling workflows, transformations are often incorporated into reusable pre-processing pipelines rather than performed manually on the complete dataset.

52 From DataFrame to NumPy

A DataFrame can be converted to a NumPy array:

```

1 X_array = X_encoded.to_numpy()
2
3 X_array

```

```
array([[ 25, 42000,  1,  0],
       [ 31, 51000,  0,  1],
       [ 28, 47000,  1,  0],
       [ 40, 68000,  0,  1],
       [ 36, 59000,  1,  0]])
```

Its shape can be inspected using:

```
1 X_array.shape
```

```
(5, 4)
```

This connects the DataFrame abstraction with the array abstraction considered previously.

The distinction is useful:

```
1 DataFrame
2   labelled columns
3   heterogeneous tabular structure
4   index-aware operations
5
6 NumPy array
7   homogeneous numerical representation
8   shape-based computation
9   no semantic column labels
```

Many machine learning libraries can accept DataFrames directly, but the underlying numerical computation is ultimately often array-based.

53 Avoiding Target Leakage

DataFrame transformations can introduce modelling errors even when the pandas code itself is correct.

One particularly important example is **target leakage**.

Suppose a feature is constructed using information that would not be available at prediction time.

The resulting model may appear to perform extremely well while relying on information it could never legitimately access in deployment.

Leakage can arise during:

- feature construction;
- aggregation;
- missing-value imputation;
- scaling;
- train/test preparation;
- joins with future information.

The relevant question is not only:

Does this transformation run correctly?

but also:

Would this information genuinely be available when a prediction must be made?

54 Train/Test Contamination

Suppose missing values are filled using a mean calculated from the complete dataset before train/test splitting.

Then information from the test observations has influenced the training transformation.

Similarly, fitting category encodings, scalers, or feature-selection procedures using all observations can contaminate model evaluation.

The safer conceptual workflow is:

split data → fit preprocessing on training data → apply to validation/test data.

This issue lies beyond pandas syntax itself, but it is essential when DataFrames are used in machine learning workflows.

55 Common DataFrame Problems

Many pandas errors are not syntax errors.

They arise because the code executes successfully while the resulting table no longer represents the intended analytical data.

55.1 Unintended Row Multiplication

A many-to-many merge may silently increase the number of rows.

After important merges, inspect:

```
1 result.shape
```

and validate expected cardinality where possible.

55.2 Unexpected Missing Values after Alignment

Series and DataFrames align by index labels.

If indexes differ unexpectedly, arithmetic or assignment may introduce missing values.

Inspect:

```
1 df.index
```

when alignment behaviour is surprising.

55.3 Incorrect Data Types

A numeric variable imported as text can alter:

- arithmetic;
- sorting;
- aggregation.

Inspect:

```
1 df.dtypes
```

rather than assuming imported types are correct.

55.4 Silent Changes in Grain

Operations such as:

```
1 groupby()
2 merge()
3 pivot_table()
4 melt()
```

can change what one row represents.

Always reconsider the dataset grain after structural transformations.

55.5 Chained Assignment

Expressions that combine multiple indexing operations can make assignment behaviour ambiguous.

For example:

```
1 df[df["income"] > 50000]["flag"] = True
```

should generally be avoided.

Explicit `.loc[]` assignment is clearer:

```
1 df.loc[
2     df["income"] > 50000,
3     "flag"
4 ] = True
```

This states directly which rows and column are being modified.

55.6 Row-Wise Iteration

Pandas provides methods for iterating through rows, but explicit row-by-row processing is often unnecessary for numerical or logical transformations.

Before writing a loop, consider whether the operation can be expressed using:

- vectorized arithmetic;
- Boolean masks;
- `.map()`;
- string accessors;
- datetime accessors;

- `groupby()`;
- `joins`.

Vectorized expressions are often clearer and better aligned with the tabular abstraction.

56 A Systematic DataFrame Workflow

When working with an unfamiliar DataFrame, a useful sequence of questions is:

1. **What does one row represent?**
2. **What does each column represent?**
3. **Which columns identify observations uniquely?**
4. **What are the data types?**
5. **Where are values missing?**
6. **Are categorical levels plausible?**
7. **Are repeated keys expected?**
8. **Will any merge change the grain?**
9. **Are indexes being used intentionally?**
10. **What information will ultimately become predictors and target?**

These questions provide a stronger foundation than memorizing isolated pandas methods.

57 Key Takeaways

A pandas DataFrame is a two-dimensional labelled tabular structure in which rows usually represent observations and columns represent variables.

Unlike NumPy arrays, DataFrames can naturally contain columns with different data types and use explicit row and column labels.

A DataFrame column is usually represented as a Series. Series and DataFrames use index labels for alignment, which distinguishes pandas operations from purely positional array computation.

`.loc[]` performs label-based selection, while `.iloc[]` performs position-based selection. Understanding the distinction between labels and positions is fundamental to reliable DataFrame manipulation.

Boolean masks allow observations to be filtered according to analytical conditions. Multiple conditions can be combined using element-wise logical operators.

Column transformations are usually most naturally expressed as vectorized operations. Pandas also provides specialized interfaces for strings, datetimes, mappings, categorical summaries, and group-level transformations.

Missing data should be identified explicitly, but their treatment is an analytical decision rather than simply a choice between `dropna()` and `fillna()`.

`groupby()` implements the split–apply–combine pattern. Aggregation reduces groups to fewer rows, while transformation can calculate group-level quantities while preserving the original number of observations.

`pd.concat()` combines tabular structures along an axis, while `merge()` combines tables according to keys. Join type, key uniqueness, cardinality, and grain determine the meaning and size of the resulting table.

Pandas indexes are not merely row numbers. They participate in label-based selection and automatic alignment.

Wide and long representations encode the same information using different table structures. `melt()`, `pivot()`, and `pivot_table()` provide mechanisms for moving between these representations.

External data can be loaded from files or databases. Import should be treated as part of data interpretation because inferred data types, missing-value conventions, and selected columns affect all downstream analysis.

Relational databases separate entities into related tables. Primary and foreign keys define those relationships, while SQL joins can construct analytical tables from them. In Northwind, the relationships among customers, orders, order details, and products provide a concrete example of one-to-many relational structure. :contentReferenceoacite:9 :contentReferenceoacite:10

SQL and pandas are complementary. SQL can efficiently retrieve and shape relational data near its source, while pandas provides a flexible in-memory environment for analysis, feature engineering, and integration with the Python machine learning ecosystem.

In machine learning workflows, DataFrames often form the interface between raw data and numerical modelling. Their labels and heterogeneous types support data preparation, while the final model representation is often numerical and array-like.

Reliable DataFrame work therefore depends less on memorizing pandas methods than on maintaining a clear understanding of **data types, labels, keys, cardinality, missingness, alignment, and the grain represented by each row.**

58 Problems

The following problems review the main concepts introduced in this chapter. They progress from basic DataFrame construction and selection to grouping, relational operations, reshaping, SQL integration, and DataFrame reasoning in machine learning workflows.

58.1 Problem 1 — Creating a DataFrame

Create the following DataFrame:

```
1 data = {  
2     "name": [  
3         "Alice",  
4         "Bob",  
5         "Carol",  
6         "David"  
7     ],  
8     "age": [  
9         25,  
10        31,  
11        28,  
12        40  
13    ],  
14     "income": [  
15        42000,  
16        51000,  
17        47000,  
18        68000  
19    ]  
20 }
```

Inspect:

- its type;
- shape;
- number of dimensions;
- column labels;
- row index;
- column data types.

Explain what one row represents and what one column represents.

58.2 Problem 2 — Series Versus DataFrame

Using the DataFrame from Problem 1, compare:

```
1 df["age"]
```

with:

```
1 df[["age"]]
```

Determine:

- the type of each result;
- the dimensionality of each result;
- the shape of each result.

Explain why the distinction between a Series and a one-column DataFrame can matter in later operations.

58.3 Problem 3 — Structural Inspection

Create a DataFrame containing:

- one integer column;
- one floating-point column;
- one string column;
- one Boolean column.

Use:

```
1 df.head()  
2 df.tail()  
3 df.info()  
4 df.describe()  
5 df.dtypes
```

Explain what information each operation provides.

Which of these operations are most useful when first inspecting an unfamiliar dataset?

58.4 Problem 4 — Label-Based and Position-Based Selection

Consider:

```
1 df = pd.DataFrame(  
2     {  
3         "name": [  
4             "Alice",  
5             "Bob",  
6             "Carol",  
7             "David"  
8         ],  
9         "score": [  
10            82,  
11            91,  
12            76,  
13            88  
14        ]  
15     },  
16     index=[  
17         10,  
18         20,  
19         30,  
20         40  
21     ]  
22 )
```

Determine the result of:

```
1 df.loc[20]  
2 df.iloc[1]  
3 df.loc[20, "score"]  
4 df.iloc[1, 1]
```

Explain why:

```
1 df.loc[1]
```

and:

```
1 df.iloc[1]
```

do not mean the same thing.

58.5 Problem 5 — Slicing with `.loc[]` and `.iloc[]`

Using the same DataFrame, compare:

```
1 df.loc[10:30]
```

with:

```
1 df.iloc[0:3]
```

Determine which rows are included in each case.

Explain the difference between the endpoint semantics of label-based and position-based slicing.

58.6 Problem 6 — Boolean Filtering

Given:

```
1 df = pd.DataFrame({
2     "name": [
3         "Alice",
4         "Bob",
5         "Carol",
6         "David",
7         "Eva"
8     ],
9     "age": [
10        25,
11        31,
12        28,
13        40,
14        36
15    ],
16    "income": [
```

```

17     42000,
18     51000,
19     47000,
20     68000,
21     59000
22 ],
23 "group": [
24     "A",
25     "B",
26     "A",
27     "B",
28     "A"
29 ]
30 })

```

Select all rows satisfying:

$$\text{age} > 30$$

and:

$$\text{income} < 65000.$$

Return only:

```

1 name
2 age
3 income

```

Use `.loc[]`.

58.7 Problem 7 — Combining Boolean Conditions

Using the same DataFrame, select observations satisfying either:

$$\text{group} = A$$

or:

`income > 60000.`

Then select observations satisfying both:

`group = A`

and:

`age ≥ 30.`

Explain why `&` and `|` are used rather than Python's scalar `and` and `or`.

58.8 Problem 8 — Membership Filtering

Using the previous DataFrame, select observations whose names belong to:

```
1 [
2   "Alice",
3   "Carol",
4   "Eva"
5 ]
```

using `.isin()`.

Then select all observations whose names do **not** belong to this collection.

58.9 Problem 9 — Creating a Derived Numerical Column

Consider:

```
1 sales = pd.DataFrame({
2   "product": [
3     "A",
4     "B",
5     "C",
6     "D"
7   ],
8   "price": [
```

```

9      10.0,
10     15.0,
11     8.0,
12     20.0
13 ],
14 "quantity": [
15     3,
16     2,
17     5,
18     4
19 ]
20 })

```

Create:

```
1 revenue
```

where:

$$\text{revenue}_i = \text{price}_i \times \text{quantity}_i.$$

Do not use an explicit Python loop.

Explain why this is a vectorized transformation.

58.10 Problem 10 — Conditional Columns

Using the DataFrame from Problem 9, create:

```
1 high_value
```

that is True whenever:

$$\text{revenue} \geq 50.$$

Then create:

```
1 revenue_category
```

with values:

```
1 High
2 Standard
```

using `np.where()`.

58.11 Problem 11 — `.assign()` and Method Chaining

Rewrite the revenue calculation using `.assign()`.

Then construct a method chain that:

1. calculates revenue;
2. retains rows where revenue is at least 30;
3. sorts the result from highest to lowest revenue.

Explain one advantage and one potential disadvantage of method chaining.

58.12 Problem 12 — Renaming and Dropping Columns

Given:

```
1 df = pd.DataFrame({
2     "p": [10, 20, 30],
3     "q": [2, 4, 5],
4     "unused": [0, 0, 0]
5 })
```

rename:

```
1 p → price
2 q → quantity
```

and remove:

```
1 unused
```

Return the transformed DataFrame.

58.13 Problem 13 — Type Conversion

Consider:

```
1 df = pd.DataFrame({
2     "id": [
3         "1",
4         "2",
5         "3"
6     ],
7     "amount": [
8         "10.5",
9         "7.2",
10        "14.8"
11    ],
12    "date": [
13        "2026-01-01",
14        "2026-01-02",
15        "2026-01-03"
16    ]
17 })
```

Convert:

- amount to a numerical type;
- date to datetime.

Inspect the data types before and after conversion.

Explain why type correctness is analytically important.

58.14 Problem 14 — Detecting Missing Values

Given:

```
1 df = pd.DataFrame({
2     "age": [
3         25,
4         np.nan,
5         31,
6         40
```

```

7 ],
8 "income": [
9     42000,
10    51000,
11    np.nan,
12    68000
13 ],
14 "group": [
15     "A",
16     "B",
17     None,
18     "B"
19 ]
20 })

```

determine:

1. which individual values are missing;
2. the number of missing values in each column;
3. the rows containing a missing income;
4. the rows with an observed income.

58.15 Problem 15 — Dropping Missing Values

Using the previous DataFrame, compare:

```
1 df.dropna()
```

with:

```

1 df.dropna(
2     subset=["income"]
3 )

```

Explain why the second operation can retain more observations.

Discuss why dropping all incomplete rows automatically may be analytically inappropriate.

58.16 Problem 16 — Simple Imputation

Replace missing income values with the median of the observed income values.

Then compare:

```
1 df["income"].median()
```

before and after the replacement.

Explain why median imputation is a modelling decision rather than merely a pandas operation.

58.17 Problem 17 — Sorting

Using an appropriate DataFrame, sort:

1. income from lowest to highest;
2. income from highest to lowest;
3. first by group alphabetically and then by income descending.

Explain what changes and what does not change when rows are sorted.

58.18 Problem 18 — Frequency Tables

Consider:

```
1 df = pd.DataFrame({  
2     "country": [  
3         "NZ",  
4         "AU",  
5         "NZ",  
6         "US",  
7         "NZ",  
8         "AU",  
9         "UK",  
10        "NZ"  
11     ]  
12 })
```

calculate:

```
1 value_counts()
2 value_counts(normalize=True)
3 nunique()
4 unique()
```

Explain what each result tells you.

58.19 Problem 19 — String Cleaning

Given:

```
1 df = pd.DataFrame({
2     "name": [
3         "  alice smith",
4         "BOB JONES ",
5         " Carol Brown "
6     ],
7     "city": [
8         "auckland",
9         "WELLINGTON",
10        "christchurch"
11    ]
12 })
```

transform the data so that:

- leading and trailing whitespace is removed;
- names are title-cased;
- cities are title-cased.

Use vectorized `.str` operations.

58.20 Problem 20 — String Filtering

Using an appropriate text column, select rows whose value:

1. contains "north", ignoring case;
2. starts with "A";
3. ends with "Ltd".

Explain why `na=False` can be useful when using `.str.contains()`.

58.21 Problem 21 — Splitting Structured Text

Given:

```
1 df = pd.DataFrame({  
2     "code": [  
3         "NZ-AKL-001",  
4         "NZ-WLG-002",  
5         "AU-SYD-003"  
6     ]  
7 })
```

split code into:

```
1 country  
2 city  
3 id
```

using:

```
1 .str.split(  
2     "-",  
3     expand=True  
4 )
```

Explain how information embedded in text has been transformed into explicit variables.

58.22 Problem 22 — Datetime Components

Given:

```
1 df = pd.DataFrame({  
2     "date": [  
3         "2026-01-15",  
4         "2026-03-21",  
5         "2026-07-04"  
6     ]  
7 })
```

convert date to datetime and create:

```
1 year
2 month
3 day
4 day_name
5 quarter
```

using .dt.

58.23 Problem 23 — Time Differences

Create two datetime columns:

```
1 start
2 end
```

and calculate:

```
1 duration
2 duration_days
```

Explain the difference between a datetime value and a timedelta value.

58.24 Problem 24 — GroupBy and Mean

Consider:

```
1 sales = pd.DataFrame({
2     "region": [
3         "North",
4         "North",
5         "South",
6         "South",
7         "South",
8         "West"
9     ],
10    "sales": [
11        100,
12        150,
13        120,
```

```

14     180,
15     200,
16     140
17 ]
18 })

```

calculate:

- mean sales by region;
- total sales by region;
- number of observations by region.

Explain the split–apply–combine interpretation of each operation.

58.25 Problem 25 — Multiple Group Aggregations

Given:

```

1 employees = pd.DataFrame({
2     "department": [
3         "A",
4         "A",
5         "B",
6         "B",
7         "B",
8         "C"
9     ],
10    "salary": [
11        60000,
12        65000,
13        70000,
14        72000,
15        68000,
16        75000
17    ],
18    "experience": [
19        2,
20        4,
21        5,
22        6,

```

```
23     3,  
24     8  
25 ]  
26 })
```

produce a summary containing, for each department:

- employee count;
- mean salary;
- median salary;
- mean experience.

Use named aggregation.

58.26 Problem 26 — Aggregation Changes Grain

Using the `employees` DataFrame, explain the grain:

1. before grouping;
2. after aggregating by department.

Why is it important to recognize that the resulting rows no longer represent individual employees?

58.27 Problem 27 — Group-Level Transformation

Add:

```
1 department_mean_salary
```

to every employee using:

```
1 groupby(...).transform(...)
```

Then create:

```
1 salary_difference
```

defined as:

$$\text{salary} - \text{department mean salary}.$$

Explain why `transform()` preserves the original number of rows while `agg()` does not.

58.28 Problem 28 — Concatenating Observations

Given:

```
1 df_a = pd.DataFrame({
2     "id": [1, 2],
3     "value": [10, 20]
4 })
5
6 df_b = pd.DataFrame({
7     "id": [3, 4],
8     "value": [30, 40]
9 })
```

combine them vertically using `pd.concat()`.

Reset the resulting index appropriately.

Explain why concatenation is appropriate here rather than a merge.

58.29 Problem 29 — Horizontal Concatenation and Index Alignment

Consider:

```
1 left = pd.DataFrame(
2     {"x": [1, 2, 3]},
3     index=["A", "B", "C"]
4 )
5
6 right = pd.DataFrame(
7     {"y": [10, 20, 30]},
8     index=["C", "B", "A"]
9 )
```

Predict the result of:

```
1 pd.concat(  
2     [left, right],  
3     axis=1  
4 )
```

Explain why the rows are aligned by index label rather than by physical position.

58.30 Problem 30 — Basic Merge

Given:

```
1 customers = pd.DataFrame({  
2     "customer_id": [1, 2, 3],  
3     "customer_name": [  
4         "Alpha",  
5         "Beta",  
6         "Gamma"  
7     ]  
8 })  
9  
10 orders = pd.DataFrame({  
11     "order_id": [  
12         101,  
13         102,  
14         103,  
15         104  
16     ],  
17     "customer_id": [  
18         1,  
19         2,  
20         1,  
21         3  
22     ],  
23     "amount": [  
24         50,  
25         80,  
26         30,  
27         120
```

```
28 ]
29 })
```

merge the two DataFrames so that each order contains the corresponding customer name.
Determine the grain of the resulting DataFrame.

58.31 Problem 31 — Join Types

Modify the previous example so that:

- one customer has no orders;
- one order refers to a customer not present in the customer table.

Compare:

```
1 how="inner"
2 how="left"
3 how="right"
4 how="outer"
```

For each result, explain which keys are retained.

58.32 Problem 32 — Cardinality

For the original customer/order relationship, determine whether the merge is:

- one-to-one;
- one-to-many;
- many-to-one;
- many-to-many.

Then use:

```
1 validate="many_to_one"
```

during the merge.

Explain which table represents the “many” side and which represents the “one” side.

58.33 Problem 33 — Unexpected Row Multiplication

Consider:

```
1 left = pd.DataFrame({
2     "key": ["A", "A"],
3     "x": [1, 2]
4 })
5
6 right = pd.DataFrame({
7     "key": ["A", "A", "A"],
8     "y": [10, 20, 30]
9 })
```

Predict how many rows result from:

```
1 left.merge(
2     right,
3     on="key"
4 )
```

Explain why.

Why can this behaviour be dangerous when repeated keys are unexpected?

58.34 Problem 34 — Index Alignment in Series

Consider:

```
1 a = pd.Series(
2     [10, 20, 30],
3     index=[
4         "A",
5         "B",
6         "C"
7     ]
8 )
9
10 b = pd.Series(
11     [1, 2, 3],
```

```
12 index=[
13     "C",
14     "B",
15     "A"
16 ]
17 )
```

Predict:

```
1 a + b
```

Explain why pandas does not calculate:

```
1 10 + 1
2 20 + 2
3 30 + 3
```

according to physical position.

58.35 Problem 35 — Missing Labels During Alignment

Consider:

```
1 a = pd.Series(
2     [10, 20, 30],
3     index=[
4         "A",
5         "B",
6         "C"
7     ]
8 )
9
10 b = pd.Series(
11     [1, 2, 3],
12     index=[
13         "B",
14         "C",
15         "D"
16 ]
17 )
```

Predict:

```
1 a + b
```

Explain why some results are missing.

58.36 Problem 36 — Duplicate Rows Versus Repeated Keys

Create a DataFrame where:

- two rows are exact duplicates;
- several rows share the same customer identifier but represent different orders.

Use:

```
1 duplicated()  
2 drop_duplicates()  
3 duplicated(subset=...)
```

Explain why repeated customer identifiers are not necessarily duplicate observations.

58.37 Problem 37 — Wide to Long Format

Given:

```
1 wide = pd.DataFrame({  
2     "customer": [  
3         "A",  
4         "B",  
5         "C"  
6     ],  
7     "sales_2024": [  
8         100,  
9         150,  
10        120  
11     ],  
12     "sales_2025": [  
13         130,  
14         160,  
15         140
```

```
16 ]
17 })
```

convert the DataFrame to long format with columns:

```
1 customer
2 year
3 sales
```

using `melt()`.

Describe the grain before and after reshaping.

58.38 Problem 38 — Long to Wide Format

Using the result from Problem 37, reconstruct a wide representation using `pivot()`.

Explain why `pivot()` requires one unique value for every combination of index and column keys.

58.39 Problem 39 — Pivot Table with Aggregation

Consider:

```
1 sales = pd.DataFrame({
2     "region": [
3         "North",
4         "North",
5         "North",
6         "South",
7         "South",
8         "South"
9     ],
10    "product": [
11        "A",
12        "A",
13        "B",
14        "A",
15        "B",
16        "B"
```

```

17 ],
18 "amount": [
19     100,
20     120,
21     80,
22     140,
23     110,
24     130
25 ]
26 })

```

create a pivot table showing mean amount for each region-product combination.

Explain why `pivot_table()` is appropriate while a simple `pivot()` is not.

58.40 Problem 40 — Reading a CSV

Suppose a file named:

```

1 transactions.csv

```

contains:

```

1 customer_id
2 transaction_date
3 amount
4 status

```

Write code that:

1. loads the file;
2. keeps only these four columns;
3. converts `transaction_date` to `datetime`;
4. inspects missing values;
5. displays the first five rows.

Explain which checks should be performed before using the `DataFrame` analytically.

58.41 Problem 41 — Writing a DataFrame

Export a transformed DataFrame to:

```
1 processed.csv
```

without writing its index.

Explain when writing the index might instead be appropriate.

58.42 Problem 42 — Northwind Schema Reasoning

Using the Northwind schema, identify the grain of the following tables:

```
1 Customers
2 Orders
3 OrderDetails
4 Products
```

Then identify the keys that relate:

```
1 Customers → Orders
2 Orders → OrderDetails
3 Products → OrderDetails
```

Explain the expected cardinality of each relationship.

58.43 Problem 43 — Northwind Customers Query

Using SQLite and pandas, write a query that returns:

```
1 CustomerID
2 CustomerName
3 Country
```

from Customers.

Load the result using:

```
1 pd.read_sql_query()
```

Then use pandas to determine the number of customers in each country.

58.44 Problem 44 — Filtering in SQL Versus pandas

Using Northwind, retrieve customers from Germany in two ways:

1. filter directly in SQL using WHERE;
2. load all customers and filter afterward using pandas.

Compare the results conceptually.

Which approach is generally preferable when the source table is extremely large?

Explain why.

58.45 Problem 45 — Northwind Orders and Customers

Write a SQL query joining:

```
1 Orders
2 Customers
```

and returning:

```
1 OrderID
2 OrderDate
3 CustomerName
4 Country
```

Explain why the resulting grain is still:

```
1 one row per order
```

even though customer information is repeated.

58.46 Problem 46 — Northwind Order Lines

Join:

```
1 OrderDetails
2 Products
```

and return:

```
1 OrderID
2 ProductID
3 ProductName
4 Quantity
5 Price
```

Load the result into pandas and create:

```
1 line_value
```

where:

$$\text{line value} = \text{Quantity} \times \text{Price}.$$

Then calculate total line value by OrderID.

58.47 Problem 47 — Northwind Multi-Table Join

Construct a query combining:

```
1 Orders
2 Customers
3 OrderDetails
4 Products
```

and returning:

```
1 OrderID
2 OrderDate
3 CustomerName
4 Country
5 ProductName
6 Quantity
7 Price
```

Before executing the query, answer:

1. What should one row represent?
2. Why will an order with several products appear in several rows?
3. Which join causes the grain to change from order-level to order-line-level?

58.48 Problem 48 — SQL Aggregation

Using Northwind, write a query that calculates the number of orders per customer.

Return:

```
1 CustomerID
2 CustomerName
3 order_count
```

Sort the result from highest to lowest number of orders.

Then describe how the same calculation could conceptually be performed using pandas after loading the relevant tables.

58.49 Problem 49 — SQL or pandas?

For each task below, decide whether you would generally prefer SQL, pandas, or whether either could reasonably be used.

1. Filter a database table containing 500 million rows to one country.
2. Join three large relational tables before transferring data to Python.
3. Experiment interactively with several derived features on a 50,000-row analytical DataFrame.
4. Calculate model residuals from arrays already in Python.
5. Aggregate billions of transactions before loading a customer-level modelling table.
6. Create exploratory categorical summaries after data have already been loaded.

Justify each choice.

58.50 Problem 50 — Preparing Predictors and Target

Given:

```
1 df = pd.DataFrame({
2     "age": [
3         25,
4         31,
5         28,
6         40,
7         36
8     ],
9     "income": [
10        42000,
11        51000,
12        47000,
13        68000,
14        59000
15    ],
16    "segment": [
17        "A",
18        "B",
19        "A",
20        "B",
21        "A"
22    ],
23    "target": [
24        0,
25        1,
26        0,
27        1,
28        1
29    ]
30 })
```

construct:

```
1 X
2 y
```

where:

- X contains the predictors;
- y contains the target.

Determine the type and shape of both objects.

58.51 Problem 51 — One-Hot Encoding

Using the DataFrame from Problem 50, encode:

```
1 segment
```

using:

```
1 pd.get_dummies()
```

with integer indicator variables.

Inspect the resulting columns.

Explain why a categorical variable may require encoding before numerical modelling.

58.52 Problem 52 — DataFrame to NumPy

Convert the encoded predictor DataFrame from Problem 51 to a NumPy array using:

```
1 .to_numpy()
```

Compare:

- shape;
- labels;
- dtype;
- structural information

between the DataFrame and the NumPy array.

What information is lost during conversion?

58.53 Problem 53 — Chained Assignment

Consider:

```
1 df[df["income"] > 50000]["high_income"] = True
```

Explain why this style should generally be avoided.

Rewrite the operation using:

```
1 .loc[]
```

so that the intended rows and target column are explicit.

58.54 Problem 54 — Vectorization Versus Row Iteration

Suppose:

```
1 df["revenue"]
```

must be calculated from:

```
1 df["price"]  
2 df["quantity"]
```

Compare:

1. explicit row iteration;
2. vectorized multiplication.

Explain why the vectorized implementation is generally preferable.

58.55 Problem 55 — Diagnose a Merge Bug

Suppose:

```
1 customers.shape == (1000, 5)  
2 orders.shape == (10000, 4)
```

and CustomerID is expected to be unique in customers.

After merging:

```
1 result = orders.merge(  
2     customers,  
3     on="CustomerID",  
4     how="left"  
5 )
```

the result unexpectedly contains 12,500 rows.

What is the most likely structural explanation?

Describe how you would diagnose the problem using:

- key uniqueness;
- duplicated keys;
- merge validation;
- row counts.

58.56 Problem 56 — Diagnose an Alignment Bug

Consider:

```
1 df = pd.DataFrame(  
2     {  
3         "value": [  
4             100,  
5             200,  
6             300  
7         ]  
8     },  
9     index=[  
10         "A",  
11         "B",  
12         "C"  
13     ]  
14 )  
15  
16 adjustment = pd.Series(  
17     [10, 20, 30],
```

```

18 index=[
19     "C",
20     "A",
21     "B"
22 ]
23 )
24
25 df["adjustment"] = adjustment

```

Predict the resulting adjustment column.

Explain why pandas assigns values according to labels rather than according to their original order.

58.57 Problem 57 — Data Leakage Through Imputation

Suppose a machine learning workflow performs:

```

1 df["income"] = (
2     df["income"]
3     .fillna(
4         df["income"].mean()
5     )
6 )
7
8 X_train, X_test, y_train, y_test = ...

```

where the mean is calculated before splitting the complete dataset.

Explain why this can introduce train/test contamination.

Describe the correct conceptual sequence.

58.58 Problem 58 — Leakage Through Aggregation

Suppose a model predicts customer churn at the beginning of each month.

A feature is created using:

```

1 total customer spending over the complete year

```

including transactions that occur after the prediction date.

Explain why this is target leakage even if the pandas aggregation is technically correct.

How should the feature be constructed instead?

58.59 Problem 59 — Grain Reasoning

Suppose three datasets have the following grains:

```
1 customers:
2   one row per customer
3
4 orders:
5   one row per order
6
7 order_lines:
8   one row per order-product combination
```

You need to build a modelling table with:

```
1 one row per customer
```

containing:

- number of orders;
- total quantity purchased;
- total expenditure.

Describe the sequence of grouping and merging operations required.

At each stage, state the grain of the intermediate DataFrame.

58.60 Problem 60 — Integrated DataFrame Reasoning

Consider the following pipeline:

```
1 result = (
2   orders
3   .merge(
4     customers,
5     on="customer_id",
```

```

6     how="left",
7     validate="many_to_one"
8 )
9 .assign(
10     order_month=lambda x:
11         x["order_date"].dt.to_period("M")
12 )
13 .groupby(
14     [
15         "customer_id",
16         "customer_name",
17         "order_month"
18     ],
19     as_index=False
20 )
21 .agg(
22     order_count=(
23         "order_id",
24         "nunique"
25     ),
26     total_amount=(
27         "amount",
28         "sum"
29     )
30 )
31 .sort_values(
32     [
33         "customer_id",
34         "order_month"
35     ]
36 )
37 )

```

Without executing the code, explain every stage.

Your answer should identify:

1. the grain of orders;
2. the cardinality expected by `validate="many_to_one"`;
3. why customer information can be merged without changing order-level grain;
4. what `order_month` represents;

5. which variables define the groups;
6. the grain after aggregation;
7. the difference between `nunique()` and a simple row count in this context;
8. why sorting does not change the grain;
9. the final meaning of one row in result.

Finally, describe the pipeline conceptually in one or two sentences.